DRIFT: DECAY RATES IN FORGETFUL BITEMPORAL MEMORY

by

Nandan Hemanth

A THESIS

Submitted to the Faculty of the Stevens Institute of Technology
in partial fulfillment of the requirements for the degree of

MASTER OF SCIENCE - APPLIED ARTIFICIAL INTELLIGENCE

Nandan Hemanth, Candidate

ADVISORY COMMITTEE

Dr. Hao Wang, Advisor Date

Dr. Joseph Helsing, Reader Date

STEVENS INSTITUTE OF TECHNOLOGY
Castle Point on Hudson
Hoboken, NJ 07030
2026

©2026, Nandan Hemanth. All rights reserved.

DRIFT: DECAY RATES IN FORGETFUL BITEMPORAL MEMORY

ABSTRACT

Long-lived language model agents accumulate memories that go stale. A fact true when written is later superseded, but retrieval keeps surfacing the old value, and the agent answers from a fact that no longer holds. Research on agent memory has concentrated on storage and retrieval; the rule deciding what memory discards has been treated as an implementation detail rather than as an object of study.

This thesis isolates that rule. DRIFT holds a bitemporal knowledge graph substrate, the fact extraction step, the retrieval function, the answer model, and the evaluation judge fixed, and varies only the forgetting policy across seven conditions spanning a no-memory floor, an append-only baseline, and five forgetting strategies drawn from current systems. The framework is evaluated on two public benchmarks that annotate the superseded answer, an annotation the staleness metric requires and that the two most widely used agent-memory benchmarks do not provide.

Four findings replicate across both benchmarks. Append-only memory measurably surfaces superseded facts, with staleness confidence intervals excluding zero. Bitemporal contradiction invalidation is statistically equivalent to the accuracy ceiling, bounded within roughly two to two and a half points on both benchmarks, while spending twelve percent fewer tokens per question and storing six and a half to fifteen percent fewer facts. Novelty-gated writing, a popular efficiency technique, increases staleness by eleven to thirteen points, because a correction is textually near-identical to the fact it corrects and is rejected as a duplicate, so it never enters memory. Structure-aware but time-blind consolidation performs significantly worse than bitemporal consolidation on real-world knowledge updates, attributing the advantage

to bitemporal semantics specifically rather than to graph structure in general.

Two methodological results accompany these findings: a configuration ablation quantifying how much of the reported efficiency depends on correctly declaring a domain's functional predicates, and a judge-reliability audit identifying a twelve-and-a-half-point accuracy artifact produced by a rotating judge pool on byte-identical answers.

The framework is released as a dependency-free, offline-verifiable Python package with a five-line integration surface, so the central question, which forgetting policy suits a given workload, can be answered empirically rather than assumed.

Author: Nandan Hemanth

Advisor: Dr. Hao Wang

Date: August 21, 2026

Department: Computer Science

Degree: Master of Science - Applied Artificial Intelligence

Acknowledgments

I am deeply grateful to my advisor, Dr. Hao Wang, for his constant support, guidance, and encouragement throughout this research, and for pushing me to see this thesis through to completion. I thank Dr. Joseph Helsing for graciously taking on the role of reader for this thesis. I am also thankful to Lisa Capestro for her constant support during the harder moments of this process. Finally, I thank my family and friends for their patience and encouragement throughout.

Table of Contents

Abstract **iii**

Acknowledgments **v**

List of Tables **x**

List of Figures **xii**

1 Introduction **1**

1.1 The Problem 1

1.2 Why the Problem Has Not Been Settled 2

1.3 Hypothesis 3

1.4 Research Questions 3

1.5 Novelty 4

1.6 Contributions 6

1.7 Organization of the Thesis 7

2 Related Work **8**

2.1 Agent Memory Systems 8

2.2 Consolidation and Its Risks 9

2.3 Write-Side Control 10

2.4 Staleness as a First-Class Problem 11

2.5 Benchmarks 11

2.6 Summary 12

3 Method **14**

3.1 The Substrate 14

3.2 The Pipeline 17

3.3 Policies Under Test 19

3.4 Metrics 21

3.5 Statistics 23

4 Experimental Setup **25**

4.1 Benchmark Selection, and a Negative Result About the Standard Benchmarks 25

4.2 The Two Benchmarks Used 27

4.3 Models and Infrastructure 30

4.4 Reproducibility 31

5 Results **33**

5.1 Headline Results 33

5.2 Effect Sizes with Paired Bootstrap Intervals 37

5.3 RQ1: Append-Only Memory Surfaces Superseded Facts 39

5.4 RQ2: Bitemporal Invalidation Is Free 40

5.5 RQ3: Bitemporal Structure Beats Structure-Blind Structure 42

5.6 Novelty Gating Causes Unrecoverable Staleness 44

5.7 Generalization Across Benchmarks 47

5.8 Ablation: How Much of the Result Is a Configuration Choice? 48

5.9 A Judge-Reliability Audit 50

5.10 RQ4: Multi-Agent Topologies 52

5.10.1 A Real-Data Pilot on the Shared Topology 54

5.11 Memory Growth 57

5.12 Per-Model Breakdown 58

6 Discussion and Conclusion 61

6.1 Why the Results Come Out This Way 61

6.1.1 Why Forgetting Can Be Free 61

6.1.2 Why Novelty Gating Fails, Structurally 62

6.1.3 Why Corroboration Count Is a Bad Currency Proxy 63

6.1.4 Why the Two Benchmarks Disagree in the Middle of the Ranking 64

6.2 Impact 65

6.2.1 For Research 65

6.2.2 For Practice 66

6.2.3 For the Broader Technology Trajectory 67

6.3 What DRIFT Means Going Forward 69

6.3.1 As an Artifact 69

6.3.2 Immediate Roadmap 70

6.3.3 Longer-Term Research Directions the Framework Enables 71

6.4 Threats to Validity 73

6.5 Limitations and Future Work 75

6.6 Conclusion 76

A Complete Per-Cell Results and Significance Matrix 1

A.1 FactConsolidation, 14 Cells 1

A.2 MQuAKE-T, 14 Cells 2

A.3 MQuAKE-T Ablation, 3 Cells 2

A.4 LoCoMo, Partial 3

A.5 Complete Pairwise Significance Matrix 3

B Figures, Dataset Schema, Reproduction, and Integration **7**

B.1 Figure Index 7

B.2 The Dataset Schema 9

B.3 Reproducing Every Number in This Thesis 10

B.4 Using DRIFT in Another Agent 10

Bibliography **13**

List of Tables

3.1 Fields carried by every fact in the bitemporal substrate. 15

3.2 The seven policies compared in this thesis. 20

4.1 Audit of `stale_answer` annotation coverage across candidate benchmarks. 26

4.2 Sizing comparison between the two benchmarks used. 29

4.3 Models and infrastructure used in every experimental cell. 30

5.1 Headline results, FactConsolidation. Bold marks the best value in a column among the policies that answer (NOMEM excluded). 34

5.2 Headline results, MQuAKE-T. Bold marks the best value in a column among the policies that answer (NOMEM excluded). 34

5.3 Effect sizes from paired bootstrap on the per-question difference. FC is FactConsolidation, MQ is MQuAKE-T. 38

5.4 Mean number of superseded facts appearing per retrieval, out of $k = 25$. 39

5.5 Cost comparison between P2 and the append-only ceiling P0. 41

5.6 P2 versus P5 on MQuAKE-T. 42

5.7 Representative questions where P5 answered from a superseded value that P2 correctly excluded. 43

5.8 Representative questions where P3's novelty gate rejected the correcting fact. 46

5.9 Policy ranking on each benchmark, by accuracy. 47

5.10 Effect of declaring `functional_predicates` on MQuAKE-T. 49

5.11 Audit of verbatim-correct answers graded incorrect. 50

5.12 The three multi-agent topologies evaluated. 52

5.13 Accuracy and staleness by policy across the three topologies, offline synthetic data. Cells are accuracy / staleness. 53

5.14 Real-data pilot, shared topology, MQuAKE-T, 96 questions, five heterogeneous extraction models. 55

5.15 Active fact count at session checkpoints. 57

5.16 Accuracy by policy, broken out by answer model. 59

6.1 Immediate roadmap, ordered by priority. 70

A.1 FactConsolidation, all 14 cells. 1

A.2 MQuAKE-T, all 14 cells. 2

A.3 MQuAKE-T ablation, 3 cells, GLM-5.2 only. 2

A.4 LoCoMo, partial, 2 of 30 cells. 3

A.5 Complete pairwise significance matrix, FactConsolidation. 4

A.6 Complete pairwise significance matrix, MQuAKE-T. 5

B.1 Figure index. 8

List of Figures

3.1 The DRIFT pipeline. Session text is converted to facts, facts pass through the bitemporal substrate where contradictions on functional predicates close validity windows, a policy performs periodic maintenance, retrieval ranks candidate facts, a second policy hook filters or reranks what the answer model sees, and the answer is scored by an independent judge alongside a token-overlap metric. 17

5.1 Accuracy and staleness by policy, MQuAKE-T. 35

5.2 Accuracy plotted against staleness for each policy, MQuAKE-T. The policy closest to the top left corner has the best combined profile. 35

5.3 Mean tokens consumed per question, by policy, MQuAKE-T. 36

5.4 All seven policies compared across every reported metric simultaneously, MQuAKE-T. 36

5.5 The no-memory floor and the append-only ceiling plotted against the five forgetting policies, MQuAKE-T. 40

5.6 Accuracy and staleness by policy across the three multi-agent topologies. 54

5.7 Mean and standard deviation of accuracy across the five independent agents in the independent topology. 54

5.8 Active fact count over the course of the 38 MQuAKE-T sessions, by policy. 58

5.9 Accuracy by policy, broken out by answer model, MQuAKE-T. 59

Chapter 1
Introduction

1.1 The Problem

Consider an assistant that is told "I live in Boston" and, three months later, "I moved to Seattle." Both statements enter the agent's memory. Both are semantically close to the query "where do I live." A retrieval function that ranks by similarity and recency will often return the older fact alongside the newer one, and the language model that composes the answer has no reliable signal telling it which of the two candidates is still true. The result is an agent that answers from a fact whose validity has already ended.

This is not a retrieval failure in the conventional sense. The correct fact was stored, and it could be found by the retrieval function precisely because it remained similar to the query. The failure is one of memory hygiene: nothing in the system marks the older fact as no longer current, and nothing removes it once a newer fact supersedes it.

The failure compounds with time. An agent that operates for a year accumulates thousands of facts, an increasing fraction of which have been superseded by later observations. Every superseded fact that is retrieved is context spent on misinformation rather than on the true state of the world, and every superseded fact that reaches the answer model is an opportunity to produce a confidently wrong statement about something the user has already corrected. As language model agents move from single-session tools toward assistants that persist across months of interaction, the volume of stale material in memory grows in proportion to the agent's

own lifetime, and the cost of ignoring it grows with it.

1.2 Why the Problem Has Not Been Settled

Production memory systems for language model agents address the storage and retrieval halves of this problem well. MemGPT organizes an agent's context as a paged hierarchy, treating the active context window as physical memory and an external store as disk [Packer et al., 2023]. Zep builds a temporal knowledge graph purpose-built for agent memory [Rasmussen et al., 2025]. Mem0 provides a scalable long-term store aimed at production deployment [Chhikara et al., 2025]. A-MEM organizes memory agentically, with a language model deciding how memories should be structured and linked [Xu et al., 2025]. In each of these systems, the treatment of information that has been superseded by later observations is secondary to the treatment of storage and retrieval, and where a forgetting mechanism exists at all, its design is chosen by the system's authors rather than measured against alternatives.

The reason this gap persists is partly methodological rather than a simple oversight. Comparing forgetting rules across published systems is confounded: the systems under comparison differ simultaneously in their substrate, their retrieval function, their extraction pipeline, their prompting, and often their underlying language model. When one system is observed to handle a stale-fact scenario better than another, the difference cannot be cleanly attributed to the forgetting rule, because four other variables changed at the same time. What is missing from the literature is not another memory architecture. It is a controlled comparison in which the forgetting policy is the only variable that changes.

This thesis supplies that comparison. The framework built to run it, called DRIFT, holds the memory substrate, the fact extraction step, the retrieval function,

the answer-generating model, and the evaluation judge fixed across every experimental condition, and varies only the rule that decides what memory keeps, discards, or hides at answer time.

1.3 Hypothesis

The central hypothesis of this thesis is stated as follows.

> **H.** Selective forgetting grounded in bitemporal structure improves an agent's answer quality and cost profile relative to both append-only memory and to structure-blind or novelty-based forgetting.

Bitemporal structure, defined precisely in Chapter 3, means that each stored fact carries two independent timelines: the interval during which the fact was true of the world, and the interval during which the agent believed it. A forgetting policy grounded in this structure can distinguish a fact that has been explicitly superseded by a later observation from a fact that is merely old, which is the distinction that a policy based on age alone, or on textual novelty alone, cannot make.

1.4 Research Questions

The hypothesis is decomposed into four research questions, each answered with evidence in Chapter 5.

RQ1. Does append-only memory, the default behavior of most deployed systems, measurably surface superseded facts at answer time?

RQ2. Among the candidate forgetting policies, which offers the best trade-off between accuracy, staleness, and computational cost?

RQ3. Does bitemporal structure specifically outperform structure-blind alternatives, or is any graph-aware consolidation strategy sufficient to obtain the same benefit?

RQ4. Do the policy rankings established under single-agent use survive multi-agent use, in which the agent that reads a fact is not necessarily the agent that wrote it?

1.5 Novelty

The contribution of this thesis is not a new memory architecture. Bitemporal modeling has a decades-long history in database research, and Zep has already applied a form of it to agent memory [Rasmussen et al., 2025]. The novelty here is experimental and lies in what a properly controlled design makes visible that an uncontrolled comparison cannot.

N1: The forgetting policy isolated as an independent variable. To the best of the author's knowledge, this is the first study that holds the substrate, the extraction step, the retrieval function, the answer model, and the judge fixed, and varies only the forgetting rule, across a taxonomy spanning no memory, append-only retention, time-based decay, contradiction-based invalidation, novelty gating, language-model consolidation, and topology-based consolidation. Published comparisons between memory systems confound the forgetting rule with several other design choices simultaneously, which makes it impossible to attribute an observed difference in behavior to the forgetting rule alone.

N2: The bitemporal-versus-structure-blind ablation. One policy in the taxonomy, referred to throughout this thesis as P5, is constructed for a single purpose:

it is structure-aware, it operates on the same knowledge graph as the winning policy, it performs the same category of consolidation, and it is deliberately denied access to valid time, so that it must resolve competing facts by structural corroboration instead of by temporal currency. This design isolates the contribution of bitemporal semantics from the contribution of graph structure in general. Without this ablation, a result showing that a bitemporal policy beats a naive baseline would be equally consistent with the more generic explanation that graph-aware reasoning of any kind helps, and would say nothing specific about temporality. With the ablation in place, the claim becomes specific, and on real-world knowledge updates it becomes decisive, as Chapter 5 reports.

N3: Novelty gating identified and measured as a failure mode. Write-side novelty gating, in which a candidate fact is rejected before it is ever stored because it is judged too similar to something already in memory, is currently promoted in the literature as an efficiency technique [Wang et al., 2026]. This thesis gives the first controlled demonstration that on any workload in which facts get corrected over time, a novelty gate trades a bounded saving in memory footprint for an unbounded liability in correctness, and demonstrates the mechanism directly from saved knowledge-graph snapshots rather than by argument alone.

N4: A negative result about the standard benchmarks. The two most widely used benchmarks for agent memory, LoCoMo and LongMemEval, carry zero annotations of the superseded answer for any of their questions, and therefore cannot support any measurement of whether an agent has correctly revised a belief [Maharana et al., 2024, Wu et al., 2025]. Identifying this gap, and identifying two benchmarks that close it, is itself a contribution to how this subfield evaluates memory revision.

Two further methodological findings emerged while reconciling experimental runs against one another rather than being planned in advance: a configuration ablation that separates the contribution of the policy itself from the contribution of a domain-specific configuration choice, and a judge-reliability audit that quantifies how much of a reported result can be attributed to noise in the evaluation procedure rather than to the policy under test. Both are reported in full in Chapter 5.

1.6 Contributions

This thesis makes the following contributions.

1. A controlled experimental framework in which the forgetting policy is the only independent variable over a fixed substrate, retrieval function, extraction step, answer model, and judge, described in Chapter 3.

2. A staleness metric kept separate from accuracy, together with the finding that the two most widely used agent-memory benchmarks cannot support it, and the identification of two benchmarks that can, described in Chapter 4.

3. Replicated evidence, on two benchmarks of different provenance, that bitemporal invalidation is free of accuracy cost, that novelty gating causes unrecoverable staleness, and that time-blind consolidation is significantly worse than bitemporal consolidation, described in Chapter 5.

4. An equivalence result rather than merely a null result. Paired bootstrap intervals bound the accuracy difference between bitemporal invalidation and the append-only ceiling to within roughly two and a half points on both benchmarks, which is a materially stronger claim than a failure to reject a null hypothesis.

5. A configuration ablation separating the contribution of the forgetting policy from the contribution of a domain declaration that the policy depends on.

6. A judge-reliability audit that establishes a noise floor for language-model-judged accuracy under a rotating pool of evaluator models, and that identifies a specific twelve-and-a-half-point artifact traced to that rotation.

7. An open, dependency-free software implementation that runs its entire test suite offline in a fraction of a second and integrates into a third-party agent in five lines of code, described in Appendix B.

1.7 Organization of the Thesis

Chapter 2 situates DRIFT among existing agent memory systems, consolidation research, write-side control techniques, and staleness-focused benchmarks. Chapter 3 describes the bitemporal substrate, the experimental pipeline, the seven policies under comparison, the metrics reported, and the statistical procedures used to test them. Chapter 4 describes the benchmark selection process, including the negative result concerning the standard benchmarks, the two benchmarks ultimately used, the models involved, and the reproducibility infrastructure. Chapter 5 reports the experimental results in full, organized around the four research questions, together with the configuration ablation and the judge-reliability audit. Chapter 6 discusses why the results take the shape they do, considers the impact of the work for research and for practice, states the threats to the validity of the conclusions in full, and closes with the limitations of the study and its conclusion. Appendix A contains the complete per-cell results and the full pairwise significance matrix. Appendix B contains the figure index, the dataset schema, the exact commands needed to reproduce every reported number, and a short example of integrating DRIFT into a third-party agent.

Chapter 2
Related Work

This chapter situates DRIFT among four bodies of work: production memory systems for language model agents, research on the risks of memory consolidation, write-side control techniques for what enters memory in the first place, and benchmarks that treat staleness as a first-class evaluation target rather than an afterthought. Each section closes by stating precisely what the cited work does not do, which is the gap this thesis addresses.

2.1 Agent Memory Systems

The last several years have produced a series of memory architectures aimed at extending a language model agent's effective context beyond what fits in a single prompt.

MemGPT introduced an operating-system-inspired design in which the agent's context window is treated as physical memory and an external store is treated as disk, with the model itself issuing function calls to page information in and out [Packer et al., 2023]. This design is primarily concerned with the mechanics of what enters and leaves the active context at a given moment, and it does not distinguish between information that is merely old and information that has been superseded by a later observation.

Zep constructs a temporal knowledge graph specifically for agent memory and is the closest architectural relative of the substrate used in this thesis [Rasmussen et al., 2025]. Zep records the interval during which a fact was valid, which is the same primitive DRIFT's substrate relies on. What Zep's published work does not do is compare alternative policies for acting on that temporal information: it demonstrates

that recording validity intervals is useful for retrieval quality, but it does not ask which of several possible rules for using those intervals to forget, invalidate, or retain facts performs best, nor does it report a staleness metric distinct from answer accuracy. This is precisely the comparison DRIFT is built to run, using a similar temporal primitive as its starting point.

Mem0 targets production deployment with a scalable long-term memory store, emphasizing operational concerns such as latency and storage cost at scale [Chhikara et al., 2025]. A-MEM organizes memory agentically, delegating decisions about memory structure and linking to a language model rather than fixing them in advance [Xu et al., 2025].

All four systems are systems contributions: each proposes an architecture and demonstrates that the architecture improves some measure of retrieval quality or operational efficiency relative to a simpler baseline. None of the four isolates the forgetting rule as a variable that can be swapped while everything else about the system is held constant, and none reports an accuracy metric alongside a metric that specifically measures whether the agent's answer reflects a fact that has already been superseded. Comparing them to one another, or to any single one of them, cannot answer the question this thesis asks, because in each comparison the forgetting rule changes together with the substrate, the retrieval function, and often the underlying model.

2.2 Consolidation and Its Risks

A separate line of work treats memory consolidation, the periodic act of rewriting, summarizing, or pruning stored memories, as a technique that can degrade an agent's behavior rather than only improve it. Zhang et al. show that memories consolidated

through repeated language-model rewriting degrade in quality over time, to the point where an agent's task utility can fall below what the same agent would achieve with no memory at all [Zhang et al., 2026]. Their headline finding, that a capable model fails on the majority of a set of previously solved reasoning problems once its memory of the solutions has been consolidated by repeated rewriting, is a direct warning against treating consolidation as a strictly beneficial operation. This finding motivates one of the design decisions of this thesis: rather than assuming that language-model consolidation is an improvement, DRIFT treats it as a policy under test, on equal footing with every other candidate. It appears in the taxonomy of Chapter 3 as policy P4, evaluated with the same metrics and the same statistical rigor applied to every other policy, rather than assumed to be a baseline improvement.

2.3 Write-Side Control

A third line of work intervenes earlier in the pipeline, at the point where a candidate fact is either admitted to memory or rejected before it is ever stored. Wang et al. propose SAGE, which uses a von Mises-Fisher density estimator over memory embeddings to classify an incoming fact as clearly novel, clearly redundant, or uncertain, routing clearly novel facts directly into memory, discarding clearly redundant ones, and reserving a more expensive language-model merge step only for the uncertain middle case [Wang et al., 2026]. The reported benefit is substantial: a reduction of roughly three and a half times in the cost of the write-time API calls needed to maintain memory. This technique is the direct model for the novelty-gating policy evaluated in this thesis as P3. Chapter 5 reports a specific and, as far as the author is aware, previously undemonstrated failure mode that follows from the design of any novelty gate, not from a particular implementation of one: a correction to an existing

fact is, by construction, textually close to the fact it corrects, which places corrections in exactly the region of embedding space a novelty gate is designed to reject.

2.4 Staleness as a First-Class Problem

A more recent line of work asks directly whether an agent can recognize that something it believes is no longer true. Chao et al. introduce STALE, a benchmark constructed to test whether an agent recognizes that a stored belief has become outdated, and identify implicit conflict, in which a new observation contradicts an old belief without explicitly announcing that it does so, as an underexplored failure mode [Chao et al., 2026]. Patel frames a closely related gap as a memory-update problem and studies remedies applied at training time rather than at inference time [Patel, 2026]. Du surveys the broader field of memory mechanisms for autonomous agents and identifies the evaluation of memory revision, meaning an agent's ability to correctly update what it believes in response to new evidence, as an open frontier for the field [Du, 2026].

The contribution of this thesis is complementary to, and orthogonal to, this line of work. Rather than asking whether a model can detect that its own memory is stale, or training a model to do so, this thesis asks a different and logically prior question: which forgetting policy, applied to the memory substrate before the model ever answers, prevents staleness from arising as a retrievable option in the first place, while holding the underlying model fixed. If the policy governing what memory retains can prevent a stale fact from ever being surfaced to the model, the model's own ability to detect staleness becomes a secondary line of defense rather than the primary one.

2.5 Benchmarks

LoCoMo and LongMemEval are the two benchmarks most commonly used to evaluate agent memory over long conversational histories, covering multi-hop and temporal reasoning questions grounded in extended dialogue [Maharana et al., 2024, Wu et al., 2025]. Neither benchmark annotates the value a stale memory would produce for any question in its question set, a gap examined in detail in Section 4.1. MemoryAgentBench contributes a split called FactConsolidation, constructed so that later facts in a session overwrite earlier facts sharing the same subject and predicate, which makes it directly usable for measuring whether an agent's answer reflects the current or the superseded value [Hu et al., 2026]. MQuAKE provides knowledge-editing instances that ship both the pre-edit and the post-edit answer for each edited fact, derived from real changes to Wikidata over time rather than from synthetic construction [Zhong et al., 2023]. GateMem studies memory governance in settings with multiple agents sharing one memory store, which is the setting most relevant to the fourth research question of this thesis, concerning whether policy rankings established under single-agent use survive multi-agent use [Ren et al., 2026].

2.6 Summary

Across these four bodies of work, a consistent pattern emerges. Systems papers propose architectures and demonstrate improvements in retrieval quality or operational cost without isolating the forgetting rule as a variable. Consolidation research demonstrates that a specific technique, language-model rewriting, can harm rather than help, but does not situate that technique within a broader controlled comparison against alternatives. Write-side control research proposes an efficiency technique without characterizing its failure modes on workloads where facts are corrected rather than

simply accumulated. Staleness-focused benchmark research asks whether a model can detect that its beliefs are outdated, a question that is complementary to, but distinct from, the question of which policy prevents staleness from being retrievable in the first place. The gap common to all four is the absence of a controlled comparison in which the forgetting policy is varied while every other component of the pipeline, including the substrate, the retrieval function, the extraction step, the answer model, and the judge, is held fixed. Supplying that comparison is the purpose of the framework described in the next chapter.

Chapter 3
Method

This chapter describes the DRIFT framework: the bitemporal substrate that every experimental condition shares, the pipeline that connects raw conversation to a judged answer, the seven forgetting policies compared, the metrics reported, and the statistical procedures used to test differences between policies. The organizing design principle, stated once here and referred to throughout the remainder of this thesis, is that the forgetting policy is the only component of the pipeline that is allowed to differ between experimental conditions. Everything upstream and downstream of the policy, including how a fact is extracted from text, how it is retrieved, how an answer is generated from it, and how that answer is judged, is identical across every condition. This is what licenses attributing any measured difference in outcome to the policy rather than to some other, uncontrolled, source of variation.

3.1 The Substrate

Every fact stored by DRIFT is represented as a subject, predicate, object triple, carrying two independent timelines and a lifecycle marker. Table 3.1 lists the fields and their meaning.

Table 3.1: Fields carried by every fact in the bitemporal substrate.

Field	Meaning
`t_valid_start`	When the fact became true of the world
`t_invalid_at`	When the fact stopped being true, null while current
`t_ingested`	When the agent learned the fact
`t_pruned`	When a policy removed the fact, null while retained
`provenance`	Which agent wrote the fact
`active`	False once the fact has been pruned

The distinction between `t_valid_start` and `t_ingested` is the defining property of a bitemporal system, as opposed to a system that records only a single timestamp per fact. A single timestamp cannot distinguish "this became true on this date" from "I learned this on this date," and the two dates frequently differ: an agent may be told today about something that became true months ago, or an agent may need to reason about what it believed at a past point in time regardless of what has since turned out to be true. Recording both timelines separately, rather than collapsing them into one, is what allows the substrate to represent the full history of a fact's status without discarding information at the point of storage.

When a new fact arrives that shares its subject and predicate with an existing fact, and that predicate has been declared *functional*, meaning it is understood to admit only one true object at a time, the previous fact's valid-time window is closed by setting its `t_invalid_at` field to the time of the new observation. Critically, this closing operation is *non-destructive*: the superseded fact remains in memory, remains addressable, and remains retrievable by any downstream component that chooses to retrieve it. No fact is ever deleted or hidden as a direct consequence of a contradiction

being detected. The only thing a contradiction does, on its own, is close a validity window.

This separation between detecting a contradiction and acting on it is the methodological core of the entire experimental design. It guarantees that the graph itself behaves identically under every experimental condition, regardless of which policy is active, because contradiction closing happens at the substrate layer, below and prior to any policy's involvement. The only thing that differs between conditions is what a policy subsequently chooses to do with a fact whose window has been closed: retain it and let it be retrieved, filter it out at retrieval time, or prune it outright. Any difference in outcome observed between two conditions can therefore be attributed to that choice, and to nothing else, because everything upstream of the policy is byte-for-byte identical between conditions.

The speech-act guard. A practical safeguard is applied before contradiction detection runs. Predicates such as *stated*, *said*, *mentioned*, and *reported* are excluded from contradiction handling even in a configuration where every predicate has been declared functional. Without this exclusion, a degenerate extraction pattern in which every line of a session is extracted as *speaker / stated / entire line* causes every fact to contradict the fact immediately before it, because every extracted triple shares the same subject and predicate and differs only in its object. This failure mode was measured directly during development: on a corpus whose true number of genuine contradictions is 133, a version of the substrate without the speech-act guard invalidated 379 of 380 facts. The failure is silent, in the sense that it produces a graph that looks superficially plausible, and it can resemble a legitimate policy result to someone inspecting only summary statistics rather than the underlying extraction. It is reported here because it is the kind of error that is easy to introduce and hard

to notice after the fact.

3.2 The Pipeline

Figure 3.1 shows the full pipeline that connects a raw session of conversation lines to a judged answer. Every experimental cell in this thesis executes the identical sequence of stages shown in the figure; the only stage that varies between cells is the policy, which is invoked at exactly three points.

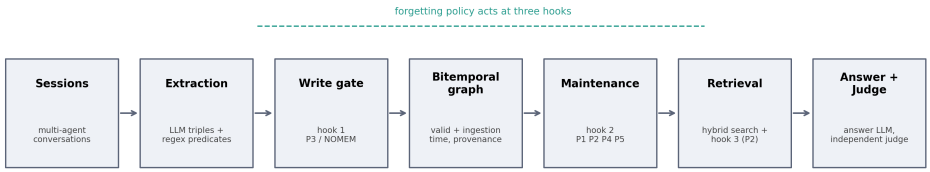


Figure 3.1: The DRIFT pipeline. Session text is converted to facts, facts pass through the bitemporal substrate where contradictions on functional predicates close validity windows, a policy performs periodic maintenance, retrieval ranks candidate facts, a second policy hook filters or reranks what the answer model sees, and the answer is scored by an independent judge alongside a token-overlap metric.

The stages, in order, are as follows. Session lines are converted into candidate facts by a language-model extraction step, backed by a deterministic regular-expression upgrade applied specifically to functional predicates, so that contradiction detection does not depend on the extractor happening to phrase two related facts identically. Each candidate fact is offered to the active policy's write hook, which may reject it outright before it ever enters the graph. Facts that are accepted enter the bitemporal knowledge graph, where a contradiction on a functional predicate closes the superseded fact's validity window in the non-destructive manner described in Section 3.1. After each session, the active policy's periodic-maintenance hook runs, which is where time-based pruning, grace-period pruning, language-model consolidation, or topology-based consolidation take place, depending on which policy is active.

When a question is asked, retrieval ranks candidate facts by a fixed hybrid score combining cosine similarity, lexical overlap, and recency, weighted 0.6, 0.3, and 0.1 respectively, and returns the top $k = 25$ candidates. The active policy's retrieval hook then has the opportunity to filter or rerank this candidate set before it is handed to the answer model. The answer model produces a response, which is then scored twice: once by a large language model judge drawn from a different model family than the answer model, and once by a token-level F1 overlap metric that does not depend on a judge at all.

A policy is defined by exactly three hooks, shown in the interface below. Adding a new policy to the framework is a matter of subclassing this interface and registering the subclass; no other component of the pipeline, including the runner, the substrate, the model providers, or the analysis code, needs to be modified or even needs to be aware that a new policy exists.

```
class Policy:
    def on_write(self, fact, graph) -> bool:
        ...  # returning False rejects the write

    def after_session(self, graph, provider, now):
        ...  # periodic maintenance, run once per session

    def on_retrieve(self, facts, now) -> list:
        ...  # filter or rerank what the answer model sees
```

3.3 Policies Under Test

Seven policies are compared, spanning a floor baseline, a ceiling baseline, and five distinct forgetting strategies. Table 3.2 summarizes each policy's mechanism and the hook or hooks it uses.

Table 3.2: The seven policies compared in this thesis.

ID	Name	Mechanism	Hooks used
NOMEM	No memory	Retrieval returns nothing; a floor baseline	`on_retrieve`
P0	No forgetting	Append-only; nothing gated, pruned, or filtered; a ceiling baseline	none
P1	TTL decay	Prune facts older than a fixed number of days by valid time	`after_session`
P2	Contradiction invalidation	Exclude superseded facts at retrieval time; prune after a seven-day grace period	`on_retrieve`, `after_session`
P3	Novelty gate	Reject writes whose neighborhood cosine similarity exceeds a threshold of 0.9	`on_write`
P4	LLM consolidation	Every third session, a language model decides which facts to keep or drop, plus a deterministic near-duplicate merge	`after_session`
P5	Topology consolidation	Resolve competing facts by structural corroboration count, deliberately denied access to valid time	`after_session`

NOMEM as a baseline. The prompt given to the answer model instructs it to respond "I don't know" whenever memory does not contain the answer to a ques-

tion. Under NOMEM, memory always fails to contain the answer, by construction, and the model's responses are almost entirely that refusal string, including in cases where the underlying language model plainly has the answer available from its own training. NOMEM therefore measures memory-gated answering behavior, not the model's parametric knowledge, and the gap between P0 and NOMEM should be read as the value memory adds under an explicit instruction to ground answers in memory rather than in general knowledge. A consequence worth stating plainly: NOMEM's staleness of exactly 0.000, reported in Chapter 5, is trivially achieved by refusing to answer at all, and staleness figures are only meaningfully comparable among policies that actually attempt an answer.

P5 as an ablation. Policy P5 is not included because it is expected to perform well. It is included specifically to test whether the advantage of bitemporal reasoning comes from temporal information in particular, or merely from any form of graph-aware consolidation. P5 builds an entity co-occurrence graph from the same underlying facts P2 has access to, finds communities of related entities using label propagation, and within each community resolves competing facts sharing the same subject and predicate by retaining whichever candidate object is corroborated by the largest number of other facts elsewhere in memory. High-degree entities, meaning entities that participate in many facts, are protected from being pruned below a floor threshold. P5 has access to every structural signal P2 has access to, with the single exception of valid time.

3.4 Metrics

Five quantities are reported for every experimental cell.

Accuracy. A language model judge, drawn from a model family different from the answer model, returns a binary verdict of correct or incorrect against the gold answer for each question. Judges are drawn from a pool of three providers with automatic fallback between them; Section 5.9 audits the consequences of this rotation for the reliability of the accuracy metric itself.

Staleness. The fraction of answers that contain the superseded value for a question, computed as a case-insensitive substring test between the model's answer and the question's annotated `stale_answer` field:

```
def staleness_hit(answer, stale_answer):
    if not stale_answer:
        return False
    return stale_answer.lower() in answer.lower()
```

Staleness can only fire on questions that carry a `stale_answer` annotation, which means a benchmark's annotation coverage places a hard upper bound on what this metric is capable of reporting for that benchmark. This is not a minor caveat; it is the subject of Section 4.1, and it is the source of a false result the author initially mistook for a finding before diagnosing its cause.

Token-F1. Unigram overlap between the model's answer and the gold answer, reported alongside accuracy because it does not depend on a judge at all. It penalizes verbose answers, which is why the two answer models used in this thesis diverge sharply on this metric while agreeing closely on judge-scored accuracy; Section 5.12 discusses this divergence and its implication for metric selection.

Cost. The mean number of tokens consumed per question at answer time, together with the count of facts the policy retains as active in memory at the end of the run.

Retrieved-invalidated. The mean number of facts appearing in a retrieval that have had their validity window closed by a contradiction. This is the mechanism variable underlying the staleness metric: it is exactly 0.00 for policy P2 by construction, because P2's retrieval hook filters such facts out before the answer model ever sees them, and it ranges from roughly 4.2 to 5.4 for the policies that do not filter on this basis.

3.5 Statistics

Four statistical procedures are used throughout Chapter 5, chosen specifically because the design compares policies that are evaluated on the identical set of questions rather than on independent samples.

- **Bootstrap 95 percent confidence intervals**, computed by the percentile method with 5,000 resamples, reported for every accuracy and staleness figure.
- **Paired McNemar exact test**, using the exact two-sided binomial distribution over discordant pairs, for comparing two policies on the same set of questions. Because every policy answers the identical question set, an unpaired test would overstate the significance of an observed difference by treating correlated outcomes as independent.
- **Paired bootstrap on the per-question difference**, used to estimate effect sizes. This procedure is a stronger form of evidence than comparing two overlapping confidence intervals computed independently for each policy, and it is what allows a failure to reject a null hypothesis to be reported as a bounded

equivalence claim rather than as an inconclusive result, a distinction developed at length in Section 5.4.

- **Spearman's rank correlation**, computed on the policy rankings produced by each benchmark, used to quantify how well a ranking established on one benchmark generalizes to a second benchmark of different provenance.

Chapter 4
Experimental Setup

This chapter describes how the benchmarks used in this thesis were selected, including a negative result that shaped the entire experimental design, the two benchmarks ultimately used and how each was adapted to DRIFT's evaluation schema, the models and infrastructure involved, and the measures taken to ensure the results are reproducible.

4.1 Benchmark Selection, and a Negative Result About the Standard Benchmarks

The staleness metric defined in Section 3.4 requires each question in a benchmark to be annotated with the value a stale memory would return for that question, referred to as the `stale_answer` annotation. Before committing to any benchmark, every reasonable candidate was audited for this annotation, with the results shown in Table 4.1.

Table 4.1: Audit of `stale_answer` annotation coverage across candidate benchmarks.

Benchmark	Conv.	Sessions	Lines	Questions	With annot.	Coverage
LoCoMo	10	272	5,882	300	0	0%
LongMemEval	n/a	n/a	n/a	150	0	0%
LongMemEval, LLM-annotated	78	156	1,830	78	1	1%
FactConsolidation	1	38	380	100	65	65%
MQuAKE-T	1	38	382	96	96	100%
Synthetic smoke (offline)	2	10	28	14	4	29%

This table exists because of a mistake made and corrected during the course of this research, and it is reported in full because the same mistake is available to anyone else working in this area. On the first experimental campaign, staleness read exactly 0.000 across all forty-two single-agent experimental cells run at that time. For a brief period this reading was interpreted as a substantive finding, namely that none of the policies under test were producing stale answers. It was not a finding. The staleness metric could not fire in principle on that data, because the annotation it depends on, `stale_answer`, does not exist anywhere in LoCoMo or in the standard LongMemEval question set. A metric that structurally cannot fire will always read zero, and a reading of zero produced by the metric's own inapplicability is indistinguishable, from the summary statistics alone, from a reading of zero produced by policies that are genuinely working well.

This is reported here as a negative result in its own right, distinct from the positive results reported in Chapter 5, because the same trap is available to any researcher measuring memory revision on LoCoMo or on LongMemEval, and because

the diagnostic that exposes it is simple: any staleness result reported for a benchmark should be accompanied by that benchmark's annotation coverage. The cross-dataset comparison tool built for this thesis prints this coverage figure automatically for exactly this reason, so that the same mistake cannot recur silently in a later run.

An attempt was also made to recover the missing annotation for LongMemEval's knowledge-update question subset using a language model annotator, prompted to identify the superseded value from the evidence turns of each conversation. This attempt produced one usable annotation out of seventy-eight questions, despite the superseded values appearing in plain, unambiguous text in the evidence turns themselves, for example a phone extension of "27:12," a dollar figure of "$350,000," or a frequency of "twice a week." The annotator's failure to recover values that are present verbatim in its own input suggests that the difficulty lies in the annotation task itself rather than in the specific prompt used, and it is the reason this thesis relies on deterministic recovery of the superseded value from a benchmark's own structure, described for each benchmark in Section 4.2, rather than on an automated annotator.

4.2 The Two Benchmarks Used

Two benchmarks satisfy the annotation requirement well enough to support this thesis's experiments: FactConsolidation, drawn from MemoryAgentBench, and MQuAKE-T, drawn from MQuAKE.

FactConsolidation. FactConsolidation is a split of MemoryAgentBench consisting of a numbered list of facts in which later facts overwrite earlier facts sharing the same subject and predicate, with the gold answer to each question always being the most recent value [Hu et al., 2026]. The superseded value for each question is recoverable without a human or automated annotator, by tracking the previous object

of whichever subject-predicate pair ultimately resolves to the gold answer. Because facts in this benchmark are presented as atomic triples rather than embedded in free-form conversation, contradiction detection is not bottlenecked by extraction quality, which makes FactConsolidation a clean setting in which to isolate the behavior of the forgetting policies themselves. Gold answers in this benchmark are deliberately counterfactual, for example the claim that rugby union was created in India, which means a language model's own parametric knowledge is actively unhelpful and cannot be used to guess the correct answer without consulting memory.

MQuAKE-T. MQuAKE-T is built by comparing Wikidata snapshots taken in April 2021 and April 2023, and retaining relations whose value changed in a way that reflects a genuine real-world update rather than a synthetic edit [Zhong et al., 2023]. Each instance in the resulting dataset ships both the pre-edit answer and the post-edit answer, so the `stale_answer` annotation is deterministic and requires no separate annotation step. MQuAKE-T was selected specifically as the generalization test for this thesis because its updates are real rather than injected, which removes the most artificial property of FactConsolidation, namely that its supersessions are constructed by the benchmark's authors rather than observed in the world, while holding the underlying task form, a fact list followed by questions about the current value, fixed between the two benchmarks.

The conversion of MQuAKE-T into DRIFT's dataset schema was sized to match FactConsolidation cell for cell, as shown in Table 4.2, so that a comparison between the two benchmarks reflects a difference in the nature of the data rather than a difference in scale.

Table 4.2: Sizing comparison between the two benchmarks used.

Property	FactConsolidation	MQuAKE-T
Sessions	38	38
Fact lines	380	382
Questions	100	96
`stale_answer` coverage	65%	100%
Policies × answer models	$7 \times 2 = 14$ cells	$7 \times 2 = 14$ cells
Question-answers pooled	200	192

Two decisions made during the conversion of MQuAKE-T are load-bearing for the validity of the results reported in Chapter 5, and both are stated explicitly here so they can be scrutinized rather than discovered later.

Randomized supersession placement. For each edited fact pair, the old value is placed in a uniformly random session i and the new value in a uniformly random later session $j > i$, rather than placing all old values into an early block of sessions and all new values into a late block. A clean chronological boundary between old and new values would let any policy that simply favors more recent sessions in general appear to win, without that policy ever needing to use bitemporal structure specifically. This is exactly the confound the ablation comparing bitemporal invalidation against structure-blind, time-blind consolidation, described in Section 5.5, is designed to rule out, and randomizing the placement of old and new values is what makes that ablation a fair test rather than a foregone conclusion.

Flat statement phrasing. Edited facts are restated in compact subject-predicate-object form, for example "The head of government of New York City is Eric Adams,"

rather than in MQuAKE's original cloze-style phrasing, for example "The name of the current head of the New York City government is." Contradiction detection depends on the extraction step producing the same subject and predicate for a fact and for the fact that later corrects it, and a longer, more grammatically varied phrasing introduces extraction variance on precisely the one join the entire experiment depends on. The original cloze phrasing is retained in the conversion script behind a configuration flag, so that the trade-off between phrasing style and extraction reliability can itself be measured in future work rather than being a silent, unexamined choice.

4.3 Models and Infrastructure

Table 4.3 lists the models used in each role in the pipeline.

Table 4.3: Models and infrastructure used in every experimental cell.

Role	Models
Answer	GLM-5.2, MiniMax-M3 (via OpenRouter)
Extraction	GLM-5.2, MiniMax-M3, gpt-oss-120b, Llama-3.3-70B, Llama-3.1-8B, Mistral Small (rotated per conversation)
Judge	gpt-oss:20b (local), gpt-oss-20b (hosted), Command-R7B (rotated per conversation)
Embedding	`nomic-embed-text`, served locally via Ollama, 768 dimensions
Retrieval depth	$k = 25$

Extraction and judging both rotate across a pool of providers, chosen specifically to spread the volume of API calls generated by a full campaign across independent rate limits rather than exhausting a single provider's quota. Section 5.9 shows that this rotation, while necessary for completing a campaign of this size in practice,

is also a confound affecting the reliability of the accuracy metric, and recommends pinning a single judge for any run whose numbers will be reported.

4.4 Reproducibility

Several measures were taken during development to ensure that every number reported in Chapter 5 can be regenerated exactly, and to reduce the practical cost of doing so.

- Every call made to a model provider is cached to disk, keyed by the SHA-256 hash of the tuple (provider, model, prompt, temperature, max tokens). An interrupted campaign resumes at zero additional cost, because every call already completed is served from the cache rather than repeated, and a rerun of a completed campaign is bit-for-bit reproducible, because the same inputs always retrieve the same cached outputs.

- Results are written one file per experimental cell, and cells that have already completed are automatically skipped on a subsequent invocation, which makes a large campaign safely resumable after an interruption of any kind.

- Failures are caught at the level of an individual cell rather than at the level of an entire campaign, so that one provider running out of quota, or one transient network failure, costs a single cell rather than terminating the whole run.

- A deterministic, offline mock provider and a deterministic, offline hashing-based embedder allow the entire pipeline to be exercised without any network access and without any API key. The complete test suite, which exercises this offline path, runs 99 tests in a fraction of a second, which means the mechanics of

the pipeline can be verified independently of, and prior to, any claim about the reported results themselves.

- The core package that implements the framework declares zero external dependencies, which removes an entire category of reproducibility failure caused by dependency version drift between the environment the results were produced in and the environment a reader later attempts to reproduce them in.

Appendix B gives the exact command sequence needed to regenerate every table and figure in Chapter 5 from a clean checkout of the codebase.

Chapter 5
Results

This chapter reports the experimental results of this thesis, organized around the four research questions stated in Section 1.4, together with two methodological results, a configuration ablation and a judge-reliability audit, that emerged while reconciling experimental runs against one another. All figures quoted in this chapter are computed from the result files produced by the experimental runs and can be regenerated exactly using the commands given in Appendix B. The complete per-cell results underlying every table in this chapter, and the full pairwise significance matrix across all twenty-one policy pairs, are given in Appendix A.

5.1 Headline Results

Tables 5.1 and 5.2 report accuracy, staleness, token cost, and active fact count for all seven policies, pooled across both answer models, on FactConsolidation and MQuAKE-T respectively. Confidence intervals are percentile bootstrap at the 95 percent level with 5,000 resamples. The sample size is 200 question-answer pairs on FactConsolidation and 192 on MQuAKE-T.

Table 5.1 is reported as two panels, one per benchmark, since every column for both benchmarks side by side does not fit within a portrait page margin at a legible size.

Table 5.1: Headline results, FactConsolidation. Bold marks the best value in a column among the policies that answer (NOMEM excluded).

Policy	Accuracy	Staleness	Tok/q	Facts
NOMEM	0.000 [.000,.000]	0.000	250	n/a
P0	**0.840** [.785,.890]	0.030 [.010,.055]	812	380
P1 (TTL 20d)	0.630 [.565,.695]	0.020 [.005,.040]	799	210
P2	**0.840** [.785,.890]	0.025 [.005,.050]	**715**	**324**
P3	0.665 [.600,.730]	**0.140** [.095,.190]	806	334
P4	0.800 [.745,.855]	0.060 [.030,.095]	814	374
P5	0.795 [.735,.850]	0.035 [.010,.060]	812	365

Table 5.2: Headline results, MQuAKE-T. Bold marks the best value in a column among the policies that answer (NOMEM excluded).

Policy	Accuracy	Staleness	Tok/q	Facts
NOMEM	0.000 [.000,.000]	0.000	233	n/a
P0	**0.969** [.943,.990]	0.052 [.021,.089]	816	382
P1 (TTL 20d)	0.870 [.823,.917]	0.026 [.005,.052]	807	251
P2	**0.969** [.943,.990]	0.036 [.010,.068]	**716**	**357**
P3	0.792 [.729,.849]	**0.161** [.109,.214]	805	351
P4	0.953 [.922,.979]	0.057 [.026,.094]	804	372
P5	0.802 [.745,.854]	**0.214** [.156,.276]	816	365

Figure 5.1 plots this table directly. Figure 5.2 places each policy in accuracy-staleness space, and Figure 5.3 plots the token-cost column.

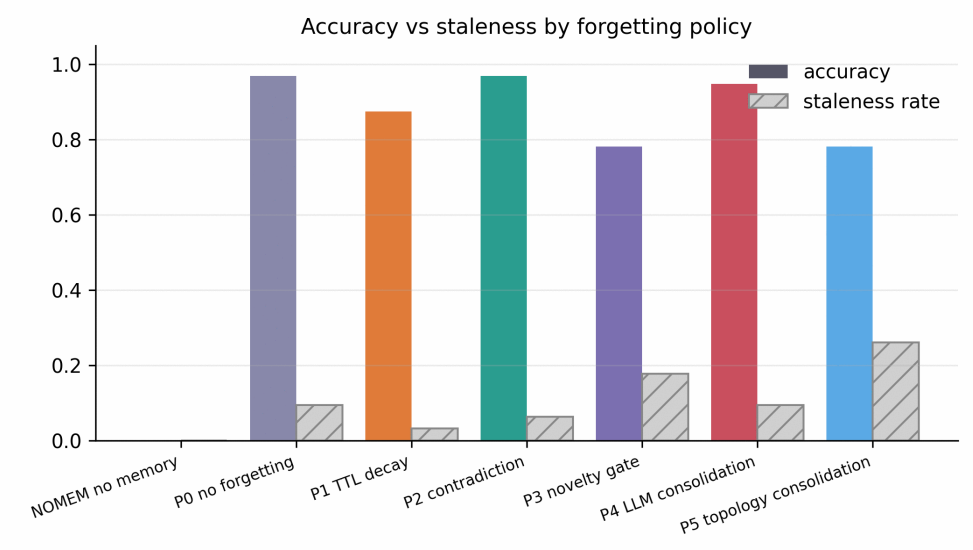


Figure 5.1: Accuracy and staleness by policy, MQuAKE-T.

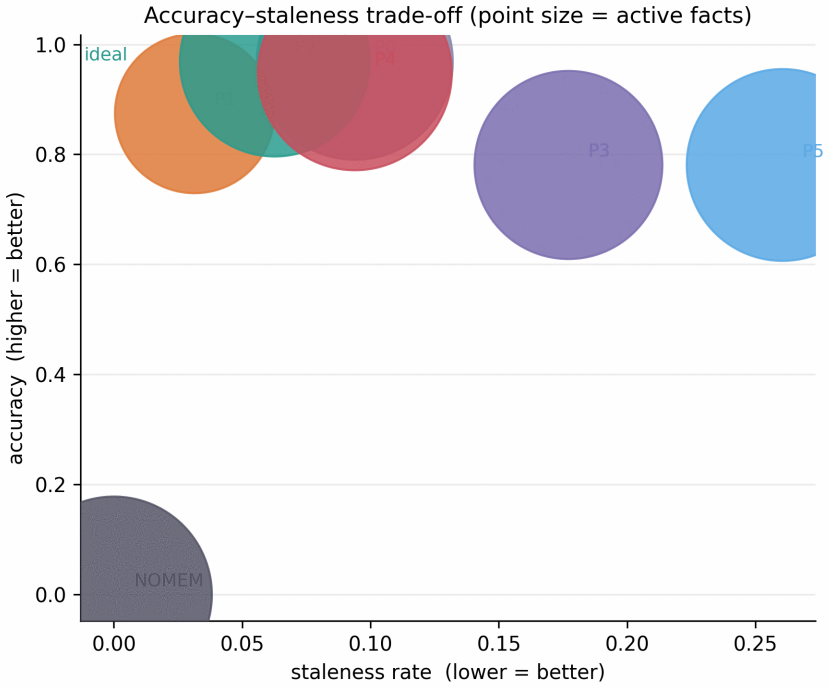


Figure 5.2: Accuracy plotted against staleness for each policy, MQuAKE-T. The policy closest to the top left corner has the best combined profile.

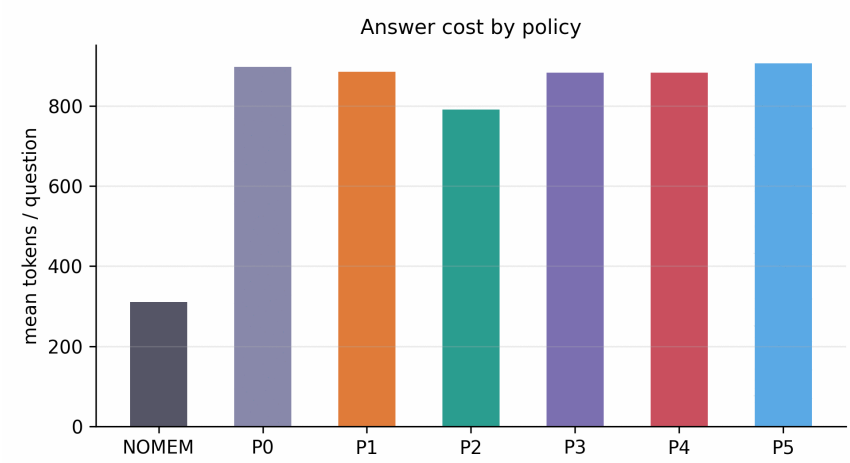


Figure 5.3: Mean tokens consumed per question, by policy, MQuAKE-T.

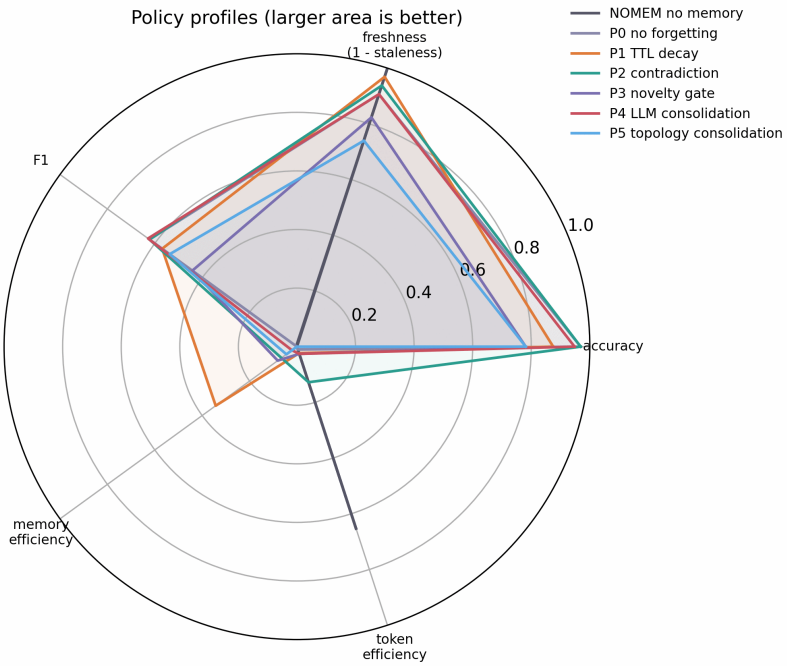


Figure 5.4: All seven policies compared across every reported metric simultaneously, MQuAKE-T.

5.2 Effect Sizes with Paired Bootstrap Intervals

Comparing two independently computed confidence intervals for overlap is a weak form of evidence, because it does not use the fact that every policy is evaluated on the same set of questions. Table 5.3 instead bootstraps the paired per-question difference directly, which is the correct estimator whenever the compared conditions share their evaluation set, as is the case for every comparison in this thesis.

Table 5.3: Effect sizes from paired bootstrap on the per-question difference. FC is FactConsolidation, MQ is MQuAKE-T.

Bench.	Comparison	Metric	Delta	95% CI	Excl. 0
FC	P2 minus P0	accuracy	+0.000	[-0.025, +0.025]	no
FC	P2 minus P0	staleness	-0.005	[-0.035, +0.025]	no
FC	P2 minus P3	accuracy	+0.175	[+0.120, +0.230]	yes
FC	P2 minus P3	staleness	-0.115	[-0.165, -0.070]	yes
FC	P2 minus P5	accuracy	+0.045	[+0.015, +0.080]	yes
FC	P2 minus P5	staleness	-0.010	[-0.040, +0.015]	no
FC	P2 minus P4	accuracy	+0.040	[+0.005, +0.080]	yes
FC	P2 minus P1	accuracy	+0.210	[+0.150, +0.270]	yes
FC	P0 minus NOMEM	accuracy	+0.840	[+0.790, +0.890]	yes
MQ	P2 minus P0	accuracy	+0.000	[-0.021, +0.021]	no
MQ	P2 minus P0	staleness	-0.016	[-0.047, +0.016]	no
MQ	P2 minus P3	accuracy	+0.177	[+0.120, +0.240]	yes
MQ	P2 minus P3	staleness	-0.125	[-0.182, -0.073]	yes
MQ	**P2 minus P5**	**accuracy**	**+0.167**	**[+0.109, +0.224]**	**yes**
MQ	**P2 minus P5**	**staleness**	**-0.177**	**[-0.240, -0.120]**	**yes**
MQ	P2 minus P4	accuracy	+0.016	[-0.016, +0.047]	no
MQ	P2 minus P1	accuracy	+0.099	[+0.057, +0.146]	yes
MQ	P0 minus NOMEM	accuracy	+0.969	[+0.943, +0.990]	yes

This is the strongest form of the evidence supporting RQ2, addressed in Section 5.4 below. The accuracy difference between P2 and the append-only ceiling is not merely a failure to detect a difference; it is bounded within roughly two and a half percentage points on FactConsolidation and roughly two and one tenth percentage points on MQuAKE-T. Reporting a bound on the difference, rather than only

reporting that a test failed to reach significance, is what turns this result into an equivalence claim, addressed in more depth in Section 6.1.1.

5.3 RQ1: Append-Only Memory Surfaces Superseded Facts

P0 staleness is 0.030, with a 95 percent interval of [0.010, 0.055], on FactConsolidation, and 0.052, with an interval of [0.021, 0.089], on MQuAKE-T. Both intervals exclude zero. Averaged across its retrievals, P0 surfaces 4.20 superseded facts per question on FactConsolidation and 4.80 per question on MQuAKE-T, out of a retrieval depth of $k = 25$.

Table 5.4 makes the underlying mechanism concrete by reporting the retrieved-invalidated metric defined in Section 3.4, which counts superseded facts appearing directly in a retrieval, prior to any answer being generated.

Table 5.4: Mean number of superseded facts appearing per retrieval, out of $k = 25$.

Policy	FC retrieved-invalidated	MQ retrieved-invalidated
NOMEM	0.00	0.00
P0	4.20	4.80
P1	2.08	3.28
P2	**0.00**	**0.00**
P3	3.91	3.94
P4	4.24	4.64
P5	4.12 to 4.16	5.31 to 5.43

P2 reads exactly 0.00 by construction, since its retrieval hook excludes any fact whose valid-time window has been closed before the answer model ever sees the retrieved candidates. P5 is the highest of any policy on MQuAKE-T, at 5.31 to 5.43,

which is the numerical signature of the mechanism analyzed in Section 5.5. RQ1 is answered affirmatively on both benchmarks: append-only memory does measurably surface superseded facts, at a rate that excludes zero under a bootstrap confidence interval on both benchmarks used.

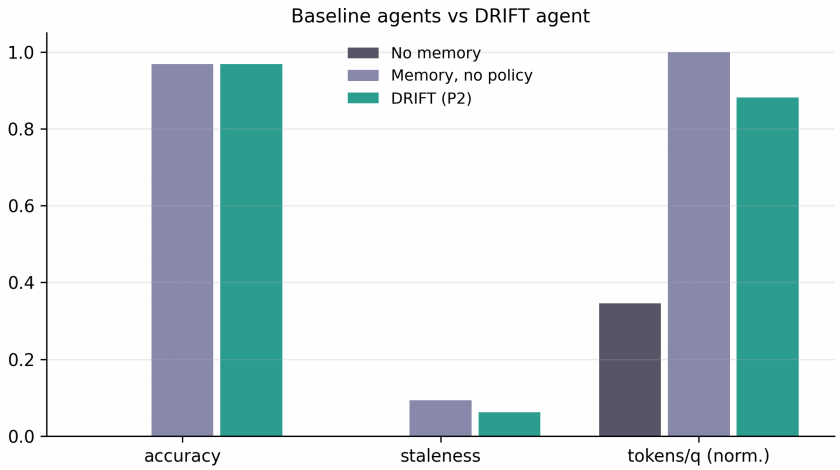


Figure 5.5: The no-memory floor and the append-only ceiling plotted against the five forgetting policies, MQuAKE-T.

5.4 RQ2: Bitemporal Invalidation Is Free

Accuracy. P2 matches P0 exactly on both benchmarks, at 0.840 against 0.840 on FactConsolidation and 0.969 against 0.969 on MQuAKE-T. The paired McNemar test returns 3 discordant pairs favoring each side on FactConsolidation ($p = 1.000$) and 2 discordant pairs favoring each side on MQuAKE-T ($p = 1.000$). The paired difference intervals reported in Table 5.3 bound the accuracy gap between P2 and P0 to within roughly ± 2.5 percentage points on both benchmarks.

Cost. Table 5.5 reports the token and memory-footprint savings P2 achieves relative to P0.

Table 5.5: Cost comparison between P2 and the append-only ceiling P0.

Benchmark	P0 tok/q	P2 tok/q	Saved	P0 facts	P2 facts	Fewer
FactConsolidation	812	715	12.0%	380	324	14.7%
MQuAKE-T	816	716	12.2%	382	357	6.5%

The token saving replicates to within two tenths of a percentage point across two benchmarks of different provenance, which is a striking degree of consistency for an effect measured on independently constructed data. The memory-footprint saving differs more between the two benchmarks, which is itself expected behavior rather than a discrepancy to be explained away: P2 prunes exactly the facts that were superseded and have since aged past their grace period, and the two source corpora simply differ in what fraction of their facts meet that description.

No other policy achieves both properties simultaneously. P1 stores substantially less memory than P0, at 210 and 251 facts against P0's 380 and 382, but loses 0.210 and 0.099 accuracy in exchange, a cost far larger than any saving P2 achieves. P3 and P5 shrink the memory footprint only modestly while losing accuracy and, as Sections 5.6 and 5.5 show, gaining substantial staleness. P4 comes close to P2 on accuracy but saves essentially nothing on either token cost or memory footprint.

RQ2 is therefore answered directly: among the seven policies compared, P2 offers the best combined accuracy, staleness, and cost trade-off, and it does so by an equivalence margin on accuracy that is tight enough to be reported as a bound rather than as an inconclusive comparison.

5.5 RQ3: Bitemporal Structure Beats Structure-Blind Structure

This section reports the ablation the experimental design exists specifically to support, and it produces the strongest result in this thesis.

P2 and P5 share the identical substrate, retrieval function, extraction step, answer model, and judge. Both are structure-aware, in the sense that both operate on the same knowledge graph and both perform a form of consolidation over it. The only difference between them is that P2 knows which of two competing facts is currently valid because it consults valid time directly, while P5 must infer which fact is current from structural corroboration count alone, having been deliberately denied access to valid time.

Table 5.6 reports the comparison on MQuAKE-T, the benchmark where the effect is largest and where the mechanism is cleanest.

Table 5.6: P2 versus P5 on MQuAKE-T.

Metric	P2	P5	Paired delta	McNemar	p
Accuracy	0.969	0.802	+0.167 [+0.109, +0.224]	34 to 2	< 0.001
Staleness	0.036	0.214	-0.177 [-0.240, -0.120]	3 to 37	< 0.001
Retrieved-invalidated	0.00	5.31 to 5.43			

P5's staleness is 5.9 times P2's staleness on this benchmark. Table 5.7 lists a representative sample of the questions where P5 returned the superseded value and P2 did not, chosen because each is independently verifiable against the public record as of the snapshot date used to construct the benchmark.

Table 5.7: Representative questions where P5 answered from a superseded value that P2 correctly excluded.

Question	Gold (current as of snapshot)	P5 answered	P2 answered
Head of state, South Korea	Yoon Suk-yeol	Moon Jae-in	Yoon Suk-yeol
Head of government, Czech Republic	Petr Fiala	Andrej Babiš	Petr Fiala
Head of state, Czech Republic	Petr Pavel	Miloš Zeman	Petr Pavel
Head of government, Malawi	Lazarus Chakwera	Peter Mutharika	Lazarus Chakwera
Head of state, Malawi	Lazarus Chakwera	Peter Mutharika	Lazarus Chakwera
Head of government, South Korea	Yoon Suk-yeol	Moon Jae-in	Yoon Suk-yeol

The mechanism. A long-serving predecessor accumulates more corroborating mentions throughout the memory graph simply by virtue of having been in office longer, and thus mentioned more often, than a recently installed successor. Corroboration-count consolidation therefore has a systematic bias toward preferring the outdated fact whenever tenure and recency are in tension. Recency alone is not a reliable proxy for correctness, since a fact can be recently observed and still wrong, but valid time specifically is a reliable proxy for currency, and that distinction between recency and validity is precisely what the bitemporal substrate encodes and what P5 has been

deliberately denied.

Why the effect is sharper on MQuAKE-T than on FactConsolidation. The same P2-versus-P5 comparison on FactConsolidation reaches $p = 0.022$ on accuracy and is inconclusive on staleness, at 3 to 5 discordant pairs with $p = 0.727$. FactConsolidation's entities are arbitrary Wikidata pairs whose corroboration is roughly uniform across the corpus, so P5's heuristic fails close to at random on that data, producing a small and comparatively noisy effect. MQuAKE-T's entities, by contrast, are real people holding real public offices, so corroboration in that corpus is systematically biased toward whoever has held the office of record for longer, and P5's heuristic therefore fails in a consistent, predictable direction rather than at random. The second benchmark did not merely replicate the finding observed on the first; it exposed the underlying mechanism in a way the first benchmark's more arbitrary entity structure could not.

RQ3 is therefore answered specifically: the advantage measured for bitemporal invalidation is attributable to bitemporal semantics themselves, and not merely to the more general property that the policy operates on graph structure. A structure-aware policy denied only temporal information performs significantly worse, at a magnitude and a level of statistical significance that leaves little room for an alternative explanation.

5.6 Novelty Gating Causes Unrecoverable Staleness

P3 exhibits 4.7 times P2's staleness on FactConsolidation, at 0.140 against 0.025, and 4.5 times P2's staleness on MQuAKE-T, at 0.161 against 0.036, both differences significant at $p < 0.001$, with paired deltas of -0.115 and -0.125 respectively.

The mechanism is structural, not statistical. A fact update is, by construction, textually near-identical to the fact it supersedes, differing typically in only a single word or phrase:

```
``Thomas Kyd was born in the city of London''   (stored)
``Thomas Kyd was born in the city of Leeds''   (the correction)
```

A novelty gate computes the cosine similarity between an incoming candidate fact and its nearest neighbors already in memory. A correction of this kind registers as a near-duplicate of the fact it corrects, and the gate rejects the write. The correction never enters memory. No downstream mechanism, whether a smarter retrieval function, a reranking step, or a more careful prompt, can recover a value that was never stored in the first place, because the failure occurs at write time, before any of those downstream mechanisms have anything to operate on.

This mechanism was verified directly from saved knowledge-graph snapshots rather than merely hypothesized. On FactConsolidation, 46 write rejections produced 24 distinct subject-predicate pairs where P0's graph recorded the update correctly while P3's graph retained only the superseded value. On MQuAKE-T, 31 write rejections produced 14 questions that P3 answered from a stale value that P0 answered correctly. Table 5.8 lists a representative sample.

Table 5.8: Representative questions where P3's novelty gate rejected the correcting fact.

Question	Gold	P3 retained
Head of state, United Kingdom	Charles, Prince of Wales	Elizabeth II
Head of state, South Korea	Yoon Suk-yeol	Moon Jae-in
Headquarters city, Scientific American	Los Angeles	New York City
Head of state, Mongolia	Khurelsukh Ukhnaa	Khaltmaagiin Battulga
Headquarters city, Apax Partners	Paris	London

This is, of the results in this thesis, the most transferable to other systems, because it follows directly from the definition of a novelty gate and from the textual form a correction necessarily takes, rather than from any detail specific to this thesis's implementation of one. The precise rate at which the failure occurs will differ across corpora with different rates of factual correction, but the failure mode itself will not, and any workload in which facts are corrected over time inherits it by construction.

The asymmetry between the saving and the liability is what makes this failure mode serious rather than merely a curiosity. The saving in memory footprint is bounded: P3 retained 334 and 351 facts against P0's 380 and 382, a reduction of 12 percent and 8 percent respectively. The liability, in contrast, is unbounded: once a correction is rejected, the wrong value persists in memory indefinitely, and it cannot be corrected by any later mechanism, because as far as the substrate is concerned,

the correction was never offered.

5.7 Generalization Across Benchmarks

Spearman's rank correlation on the policy ranking across the two benchmarks is 0.891, computed over all seven policies. Table 5.9 shows the ranking on each benchmark side by side.

Table 5.9: Policy ranking on each benchmark, by accuracy.

Policy	FC rank	MQ rank	Stable
P0	1	1	yes
P2	2	2	yes
P4	3	3	yes
P5	4	5	no
P3	5	6	no
P1	6	4	no, moves two places
NOMEM	7	7	yes

The most informative row in this table is P1's. On MQuAKE-T, every question by construction concerns a fact that has changed, so the gold answer is always the recent value, and time-based pruning removes mostly facts that have genuinely been superseded, which is largely harmless. On FactConsolidation, only 65 percent of questions concern a fact that has changed, so time-based pruning also removes a substantial number of stable, still-answerable facts, which is directly destructive to accuracy. P1's rank, in other words, is a property of how much of the target workload consists of updates, rather than an intrinsic property of time-based pruning as a strategy.

The top and the bottom of the ranking are stable across both benchmarks; the middle of the ranking is workload-dependent. This should not be read as a weakness of the study. It is the study's own central argument, demonstrated directly: which forgetting policy is best is an empirical question about the target workload rather than a question with a universal answer, and P1 shifting by two ranking places when the fraction of update-relevant questions changes is the clearest available demonstration of that argument within the data collected for this thesis.

5.8 Ablation: How Much of the Result Is a Configuration Choice?

The `functional_predicates` setting declares to the substrate which predicates in a given domain admit only a single true object at a time, which is the information contradiction detection relies on. FactConsolidation required this setting to be configured as "all predicates," a choice made while directly inspecting that dataset, which makes it the single largest threat to the external validity of the results in this thesis. To quantify the consequence of this choice, MQuAKE-T was rerun with the setting left at the substrate's built-in default, a set of predicates drawn from personal-conversation contexts that happens to cover none of MQuAKE-T's actual relations. Under this default, invalidations in the graph drop from 59 to 1, which effectively switches P2's contradiction-detection mechanism off.

Table 5.10: Effect of declaring `functional_predicates` on MQuAKE-T.

Cell	Declared	Not declared	Change
P2 accuracy	0.969	0.969	none
P2 tokens per question	641	733	+14.4%
P2 active facts	357	382	+7.0%
P2 retrieved-invalidated	0.00	0.00	none
P0 accuracy	0.969	0.969 (corrected, see Section 5.9)	none
P3 accuracy	0.802	0.802	none, as expected
Graph invalidations	59	1	mechanism off

The declaration buys P2 none of its accuracy and all of its efficiency. This is a more precise and more useful result than either of the two simpler alternatives, that the setting matters or that it does not. It decomposes P2's overall benefit into two separable parts: P2 is safe to enable without tuning, because an incorrectly configured P2 degrades gracefully to the behavior of append-only memory rather than to something worse, but P2 only pays for itself, in the sense of reducing token cost and memory footprint, when the domain's functional predicates have actually been declared. That is a specific, actionable deployment instruction derived directly from measurement rather than from intuition.

5.9 A Judge-Reliability Audit

While reconciling the ablation run described in Section 5.8 against the main MQuAKE-T run, the ablation's P0 cell was found to have produced 96 of 96 byte-identical answers to the main run's P0 cell, and yet was scored 0.844 against the main run's 0.969. Twelve verdicts flipped between the two runs, all in the same direction, on answers of the following character:

```
Question:  What is the name of the current head of
the Australia government?
Gold:  Anthony Albanese
Answer:  Anthony Albanese   (judged INCORRECT)
```

To quantify how often this occurs, every experimental cell was audited for answers whose normalized form contains the gold answer verbatim and yet were graded incorrect by the judge. Table 5.11 reports the result.

Table 5.11: Audit of verbatim-correct answers graded incorrect.

Scope	Verbatim-correct answers	Judged incorrect	Rate
FactConsolidation, all cells	515	3	0.6%
MQuAKE-T, all cells	782	12	1.5%
MQuAKE-T ablation, P0 cell alone	93	14	15.1%

The baseline judge false-negative rate across the full campaign is roughly 1 to 2 percent, which is small enough that it does not change any conclusion reported

elsewhere in this thesis. One specific cell, however, happened to draw a comparatively unreliable member of the three-provider judge fallback pool and lost 12.5 accuracy points on answers that were, by direct string comparison, correct. The ablation's reported P0 accuracy of 0.844 is therefore an artifact of judge assignment rather than a property of the policy; the true value, reported in Section 5.8 and Table 5.10, is 0.969.

Three consequences follow from this audit.

1. Any reported gap between two policies smaller than roughly three percentage points should be treated as inside the noise floor established by this audit and not treated as interpretable evidence of a genuine difference. This places the comparison between P2 and P4 on MQuAKE-T, with a delta of +0.016 and $p = 0.549$, outside the range this thesis is willing to interpret, while leaving the results reported in Sections 5.5 and 5.6, at 16.7 and 17.7 points respectively with $p < 0.001$ in both cases, far outside the noise floor and therefore fully interpretable.

2. Judge rotation across a fallback pool is simultaneously a resilience mechanism and a confound. The pool exists precisely so that one provider running out of quota does not terminate an entire campaign, but its existence also means two experimental cells can, in principle, be graded by two different underlying judge models without that fact being visible in the summary statistics. The practical recommendation is to pin a single judge for any run whose numbers will be reported, or, at minimum, to record which judge model graded each individual record so that this kind of artifact is auditable after the fact rather than requiring archaeology to uncover.

3. The verbatim-match audit performed here is inexpensive, requiring on the order

of ten lines of code, and it caught a 12.5-point artifact that would otherwise very plausibly have been written up as a genuine policy effect.

This audit is reported here in full, including the specific cell in which the artifact occurred, because it is a class of error that is easy to commit and correspondingly hard to detect from summary statistics alone, and because the practice of using a language model as a judge is now standard across this subfield of research.

5.10 RQ4: Multi-Agent Topologies

The multi-agent harness built for this thesis runs the same seven policies across three topologies, all operating over the same substrate and the same policy implementations used in the single-agent experiments. Table 5.12 describes the three topologies.

Table 5.12: The three multi-agent topologies evaluated.

Topology	Structure	What it isolates
Shared	Five writer agents round-robin into one shared graph, each fact tagged with the identity of the agent that wrote it	Write contention: a contradiction can now arrive from a different agent than the one that wrote the original fact
Independent	Five agents, each with a private graph, each seeing a seeded random 80 percent of sessions	Robustness under partial observation of the underlying data
Orchestrator	One shared graph, with a router language model assigning each incoming question to one of five specialist worker agents	The production coordinator-worker pattern, in which the agent that reads a fact is not the agent that wrote it

Table 5.13 reports results on synthetic data using the offline mock provider, with 14 questions per cell.

Table 5.13: Accuracy and staleness by policy across the three topologies, offline synthetic data. Cells are accuracy / staleness.

Policy	Shared	Independent	Orchestrator	Cross-writer inval.
NOMEM	0.000 / 0.000	0.000 / 0.000	0.000 / 0.000	5
P0	0.714 / 0.286	0.667 / 0.286	0.714 / 0.286	5
P1 (TTL 14d)	0.143 / 0.000	0.143 / 0.000	0.143 / 0.000	3
P2	**0.929 / 0.000**	**0.881 / 0.000**	**0.929 / 0.000**	5
P3	0.714 / 0.286	0.667 / 0.286	0.714 / 0.286	5
P4	0.714 / 0.286	0.667 / 0.286	0.714 / 0.286	5
P5	0.500 / 0.143	0.548 / 0.095	0.500 / 0.071	4

P2 leads all seven policies in all three topologies and holds staleness at exactly 0.000 in all three. Every shared-topology invalidation observed arrives cross-writer, meaning the fact that closed a validity window was written by a different agent than the one that wrote the original fact, which is direct evidence that structure-aware forgetting of this kind is writer-agnostic by construction rather than by any special-case handling.

This result should be read as validation of the multi-agent harness rather than as a settled finding. The sample size is 14 questions using a deterministic mock provider rather than a real language model, and it covers the full seven-policy, three-topology design without yet answering whether that design holds under real, heterogeneous multi-model extraction. Section 5.10.1 reports a partial real-data pilot that probes exactly this question on one topology, and finds a real boundary condition. Of the four research questions posed in Chapter 1, RQ4 remains the one with the weak-

est evidentiary support, and Chapter 6 discusses completing it as the highest-priority item of future work.

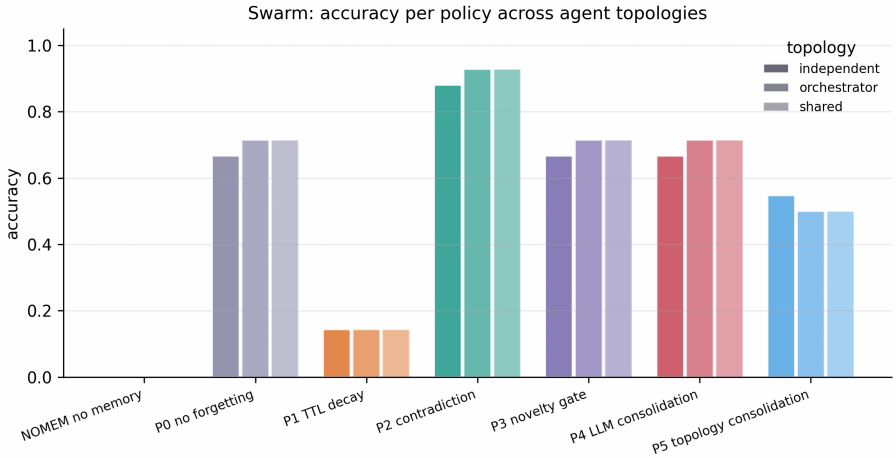


Figure 5.6: Accuracy and staleness by policy across the three multi-agent topologies.

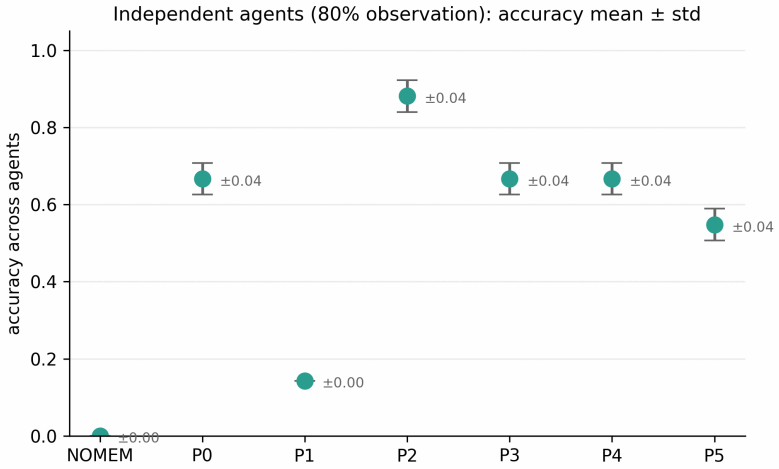


Figure 5.7: Mean and standard deviation of accuracy across the five independent agents in the independent topology.

5.10.1 A Real-Data Pilot on the Shared Topology

A partial real-data campaign was run on the shared topology using MQuAKE-T and five heterogeneous extraction models, one per writer agent: GLM-5.2, MiniMax-M3,

Gemini-3.1-Flash-Lite, Nemotron-3-Ultra, and Hunyuan, with GLM-5.2 as the shared answer model. Owing to the wall-clock cost of the P4 and P5 policies at this sample size, the pilot covers five of seven policies (NOMEM, P0, P1, P2, P3) on one of three topologies rather than the full matrix. Table 5.14 reports the results.

Table 5.14: Real-data pilot, shared topology, MQuAKE-T, 96 questions, five heterogeneous extraction models.

Policy	Accuracy	Staleness	Tok/q	Invalidations	Active facts
NOMEM	0.000	0.000	156	0	140
P0	0.333	0.271	710	0	140
P1 (TTL 20d)	0.281	0.135	729	0	79
P2	0.365	0.271	710	0	140
P3	0.365	0.271	710	0	140

The invalidation column is the finding. In the single-agent runs, one consistent extraction model per cell, P2 closes dozens of valid-time windows on MQuAKE-T (Section 5.3). Here, with five different models extracting from the same sessions, P2 records zero invalidations, and its retained-fact count, 140 active and 0 pruned, is identical to P0's. P2 and P3 track P0 to within 3.2 accuracy points, a gap the judge-reliability audit in Section 5.9 already established as inside single-provider judge noise at this sample size, so the honest reading is that all three are statistically indistinguishable in this cell: bitemporal invalidation did not engage.

The mechanism is visible directly in the saved graph. Of 140 stored facts spanning 30 distinct predicate strings, the same underlying relation is extracted under multiple surface forms depending on which model wrote it: `head_of_government` (28 occurrences), `is_head_of_government_of` (9), and `head_of_government_of` (7) all co-occur for what MQuAKE-T's schema treats as one functional predicate, and the same

pattern appears on `citizen_of` (32) versus `is_citizen_of` (6) versus `citizenship` (5). P2's contradiction detection matches on subject and predicate string; a correction extracted under a different surface form than the fact it corrects never triggers a match, so the valid-time window never closes. Only 3 of 137 subjects in the graph carry more than one fact at all, each exactly 2, versus the dozens of multi-fact subjects typical of the single-extractor single-agent runs, consistent with the same real-world relation being scattered across string variants rather than consolidated onto one.

P1 is the one policy in this pilot whose staleness reduction survives extraction heterogeneity, 0.135 versus P0's 0.271, because time-to-live pruning keys on `t_valid_start` alone and never needs two facts to agree on a predicate string to act. That protection is not free: P1 also prunes facts that were never superseded, and its accuracy, 0.281, is the lowest of the four non-floor policies in this pilot.

This pilot does not overturn the single-agent findings above; it identifies a boundary condition on them. P2's advantage over P0 was established, and holds, when one extractor is responsible for the graph's predicate vocabulary. It is not yet established for the heterogeneous-extractor case this pilot probes, and the mechanism above, predicate-string fragmentation defeating exact-match contradiction detection, is a directly observed explanation rather than a speculative one. A canonicalization step ahead of the write-time match, mapping extracted predicate strings to a fixed schema or matching on embedding similarity rather than string equality, is the natural fix and is noted as future work in Chapter 6. The remaining sixteen cells of the full twenty-one-cell matrix, both remaining topologies plus P4 and P5 on the shared topology, were not completed in the time available for this thesis; Table 5.13 above remains the only evidence on the full seven-policy, three-topology design, and it is offline-synthetic.

5.11 Memory Growth

Table 5.15 reports the count of active facts in memory at five checkpoints across the 38 sessions of each benchmark, using the GLM-5.2 answer model.

Table 5.15: Active fact count at session checkpoints.

	FactConsolidation				
Policy	s1	s10	s20	s30	s38
P0	10	100	200	300	380
P1	10	100	200	210	210
P2	10	100	189	270	324
P3	10	98	185	273	334
P4	10	96	195	294	374
P5	10	100	198	291	365
	MQuAKE-T				
Policy	s1	s10	s20	s30	s38
P0	1	74	169	276	382
P1	1	74	169	209	251
P2	1	74	165	266	357
P3	1	74	164	264	351
P4	1	74	166	266	369
P5	1	74	168	271	365

Two observations follow from this table. P1's curve flattens abruptly once its time-to-live horizon is reached, which is the direct cause of its accuracy collapse reported in Section 5.1: it is discarding answerable facts wholesale once they cross

the age threshold, rather than selectively discarding only facts that have actually been superseded. P2's curve, by contrast, tracks P0's closely for most of the run and diverges only gradually, which is exactly the shape one would expect from a policy that removes only what has genuinely become invalid rather than removing by a blunt criterion such as age.

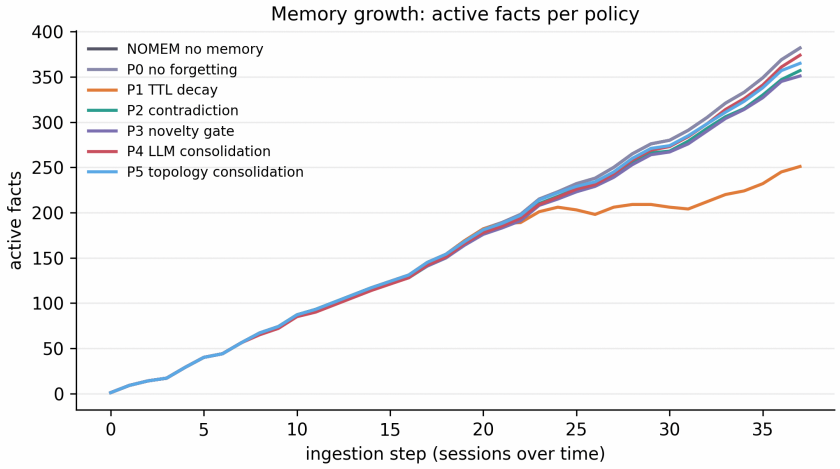


Figure 5.8: Active fact count over the course of the 38 MQuAKE-T sessions, by policy.

5.12 Per-Model Breakdown

Table 5.16 reports accuracy broken out by the two answer models used in this thesis, GLM-5.2 and MiniMax-M3, to check whether the policy effects reported above are an idiosyncrasy of a single answer model.

Table 5.16: Accuracy by policy, broken out by answer model.

Policy	FC GLM-5.2	FC MiniMax-M3	MQ GLM-5.2	MQ MiniMax-M3
NOMEM	0.000	0.000	0.000	0.000
P0	0.860	0.820	0.969	0.969
P1	0.650	0.610	0.865	0.875
P2	0.850	0.830	0.969	0.969
P3	0.680	0.650	0.802	0.781
P4	0.820	0.780	0.958	0.948
P5	0.820	0.770	0.823	0.781

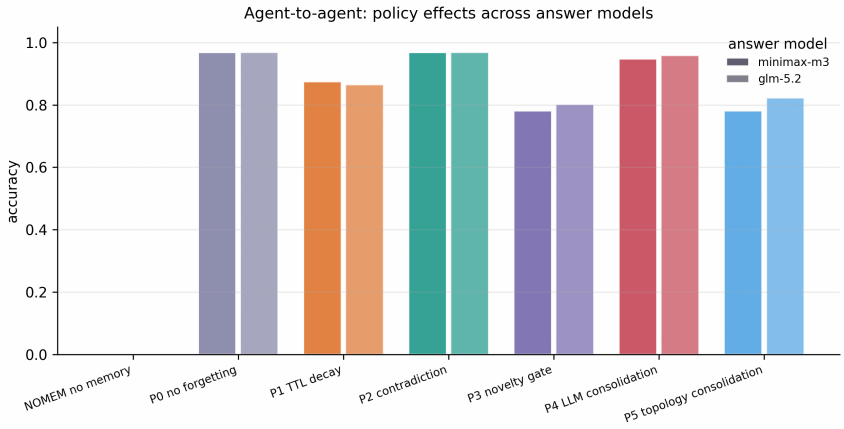


Figure 5.9: Accuracy by policy, broken out by answer model, MQuAKE-T.

Both answer models rank all seven policies in the identical order. Token-F1, not shown in this table, diverges sharply between the two models, for example 0.973 against 0.622 for P0 on MQuAKE-T, because MiniMax-M3 tends to answer more verbosely and token-F1 penalizes length directly. Accuracy, which is scored semantically by a judge rather than by lexical overlap, agrees closely between the two models despite this divergence in verbosity. This is itself a small methodological observation

worth stating explicitly: token-F1 should not be relied upon as a primary metric when comparing models with substantially different verbosity profiles, precisely because it can disagree with a semantic judgment for reasons unrelated to correctness.

Chapter 6
Discussion and Conclusion

This chapter interprets the results reported in Chapter 5, discusses their significance for research and for practice, situates the DRIFT framework as an artifact going forward, states the threats to the validity of the conclusions in full, describes the limitations of the study and the work they motivate, and closes with a summary conclusion.

6.1 Why the Results Come Out This Way

6.1.1 Why Forgetting Can Be Free

The common intuition that forgetting must cost accuracy rests on an implicit assumption: that all stored information has non-negative expected value at the moment it might be retrieved. Bitemporal invalidation violates that assumption in a specific and identifiable way. It removes only facts whose valid-time window has been closed, meaning facts that a later observation has explicitly contradicted. Such facts have negative expected value at retrieval time, because surfacing them to the answer model can only produce a stale answer; there is no scenario under this policy's design in which retrieving an invalidated fact helps.

This explains the shape of the result reported in Section 5.4. P2 does not trade accuracy for efficiency in the way a naive intuition might predict. It removes information that was already harmful to retrieve, so the efficiency gain arrives without a corresponding accuracy cost, and the paired bootstrap interval bounding the P2-versus-P0 accuracy gap to within roughly two and a half points is the quantitative

expression of that fact. Time-to-live decay, policy P1, removes facts by age instead, a criterion that is uncorrelated with whether a fact has actually been superseded. It therefore destroys useful and harmful information indiscriminately, and it pays for whatever memory savings it achieves in a substantial loss of accuracy, as Section 5.1 reports.

The general principle this result supports is that a forgetting policy is free, in the sense of costing nothing in accuracy, exactly to the extent that its removal criterion correlates with the harmfulness of the information being removed, rather than with the information's age or its textual redundancy relative to other stored facts.

6.1.2 Why Novelty Gating Fails, Structurally

A novelty gate makes an inference from textual similarity to informational redundancy: if a candidate fact looks like something already in memory, the gate infers that storing it adds nothing new. That inference is sound for genuine duplicates, restatements of a fact that add no new information, but it fails precisely for corrections, because a correction is, almost by definition, a minimal edit of the thing it corrects. The gate's discriminative signal, textual similarity, is inversely aligned with exactly the case that matters most for an agent that needs to stay current: the case in which a new fact should overwrite an old one.

This is not a problem that can be solved by tuning the gate's similarity threshold. Lowering the threshold admits more genuine duplicates into memory without gaining any ability to distinguish a correction from a restatement, because the two are textually indistinguishable at the level of an embedding. Distinguishing them requires consulting exactly the structural information a novelty gate does not consult: whether the candidate fact shares a subject and a predicate with an existing fact and

differs specifically in the object. That comparison is the bitemporal contradiction test described in Section 3.1, not a similarity threshold.

The two policies are therefore not simply two different points along a single trade-off curve between memory size and accuracy. They are, in a precise sense, inverses of one another. Both shrink the memory footprint relative to append-only retention, but P2 preserves an agent's ability to be corrected by a later observation, while P3 destroys that ability, silently and irreversibly, at exactly the moment a correction is offered.

6.1.3 Why Corroboration Count Is a Bad Currency Proxy

P5's heuristic asks, in effect, which of two competing values the rest of memory agrees with. On a corpus describing real entities, agreement of this kind accumulates with tenure: a head of state who has served for eight years appears in more contexts, and is mentioned by more other facts, than a successor who took office only recently. The heuristic therefore carries a systematic bias toward the past, and it carries that bias in exactly the setting where distinguishing the past from the present matters most.

The general statement this supports is that structural corroboration measures how established a fact is, not whether that fact is currently true. Those two quantities happen to be positively correlated for facts that are stable over long periods, which is most facts most of the time, and they become negatively correlated specifically for facts that are in the process of being revised, which is precisely the subset of facts a forgetting policy exists to handle correctly. A heuristic that performs well on the common case and fails on the case the policy was built for is a heuristic whose apparent success can be misleading if evaluated only on aggregate statistics.

6.1.4 Why the Two Benchmarks Disagree in the Middle of the Ranking

FactConsolidation and MQuAKE-T differ along one axis that turns out to matter for the ranking reported in Section 5.7: the fraction of questions in each benchmark that concern a fact which changed over the course of the sessions. On MQuAKE-T this fraction is 100 percent, by construction. On FactConsolidation it is 65 percent.

Policies whose removal criterion is aligned with supersession, of which P2 is the clearest example, are insensitive to this ratio, because they remove exactly the facts that changed regardless of how many such facts there are in a given benchmark. Policies whose criterion is unaligned with supersession, of which P1 is the clearest example because it removes by age rather than by validity, are highly sensitive to this ratio: when nearly every question concerns a recent change, removing old facts is largely harmless, because the old facts being removed are mostly the superseded ones anyway, but when a substantial minority of questions concern facts that never changed at all, the same removal strategy becomes directly destructive, because it removes stable, still-answerable facts along with the superseded ones.

This yields a practical diagnostic that follows directly from the data: before selecting a forgetting policy for a given deployment, estimate the update ratio of the target workload, meaning the fraction of stored facts that are likely to be corrected over the agent's operational lifetime. A high update ratio makes even a crude, unaligned policy look acceptable in evaluation. A low update ratio exposes the same policy's weaknesses clearly. Section 6.2.2 develops this observation into a concrete recommendation.

6.2 Impact

6.2.1 For Research

This thesis contributes a measurement instrument to a subfield that has, until now, largely reasoned about forgetting policies by assertion rather than by controlled comparison. The comparison DRIFT performs is not recoverable from published results alone, because published memory systems differ from one another along several dimensions simultaneously, including substrate, retrieval function, extraction pipeline, and underlying model, which makes isolating the contribution of any single dimension, including the forgetting rule, impossible after the fact. Making the forgetting rule separable from these other dimensions is a precondition for the field to accumulate reliable knowledge about it, in the way that holding other variables fixed is a precondition for any controlled experiment.

This thesis also contributes a correction to the evaluation practice of this subfield. The finding, reported in Section 4.1, that LoCoMo and LongMemEval carry zero annotations of the superseded answer for any question, and therefore cannot support any measurement of whether an agent correctly revises a belief, affects the interpretation of any published work that claims to evaluate memory updating using either benchmark. The associated recommendation, that any staleness or belief-revision result be reported alongside the annotation coverage of the benchmark it was measured on, costs essentially nothing to follow and would prevent a specific, easy, and previously undetected class of error.

The judge-reliability audit reported in Section 5.9 contributes a noise floor for the increasingly common practice of using a rotating pool of language models as an evaluation judge, for reasons of cost and rate-limit management. This thesis shows that such rotation can introduce a confound of up to 12.5 accuracy points between

two runs that produced byte-identical answers, and supplies a ten-line detector that any project using a similar evaluation setup can adopt directly.

Finally, this thesis contributes a named failure mode with an identified mechanism. Write-gate-induced staleness, described in Section 6.1.2, now has a name, a controlled measurement on two independent benchmarks, a demonstrated mechanism traced through saved graph snapshots, and reproducible evidence. Future work proposing write-side control techniques for agent memory can treat this failure mode as a constraint to be explicitly designed against, rather than rediscovering it independently.

6.2.2 For Practice

The engineering recommendation that follows from this thesis is narrow, measured, and falsifiable rather than aspirational:

> If an agent tracks facts that get corrected over time, use bitemporal contradiction invalidation, declare the domain's functional predicates explicitly, and do not use a novelty write gate. If the agent's facts are append-only and never revised, the question this thesis addresses does not arise, and any of the simpler policies compared here is adequate.

Three properties make this recommendation actionable rather than merely a suggestion.

It is free at the margin. P2 is statistically equivalent to append-only memory on accuracy while reducing token spend by roughly 12 percent on both benchmarks tested. In a deployed assistant serving a large number of sessions over time, this token saving is a direct, ongoing cost reduction with no measured quality cost attached to it.

It fails safe. The ablation reported in Section 5.8 shows that a P2 deployment with an incorrectly or incompletely configured `functional_predicates` setting degrades to the behavior of append-only memory, not to something worse. An engineer adopting P2 can therefore enable it without a lengthy tuning campaign and without exposing the deployment to a downside risk beyond what append-only memory already carries.

It names the one thing that must be configured. The `functional_predicates` setting is the single configuration choice that determines whether P2 pays for itself in reduced token and memory cost. This is a plain, inspectable list of predicate names specific to a given domain, not an opaque hyperparameter that requires a search procedure to tune correctly.

The framework itself is built to lower the cost of taking this recommendation on faith rather than verifying it. Exporting a workload into DRIFT's dataset schema, described in Appendix B.2, writing a short configuration file, and running four policies against it is an afternoon of work, and the resulting comparison is specific to the deployment in question rather than borrowed from the benchmarks used in this thesis.

6.2.3 For the Broader Technology Trajectory

Language model agents are moving from single-session tools toward persistent systems that accumulate state across months of continuous operation. Three consequences follow from the results of this thesis for that broader trajectory.

Memory hygiene is a correctness problem, not only an efficiency problem. The dominant framing of memory management in current systems is a cost framing: context is expensive to retain and to retrieve, so the goal is to store less of it. This thesis shows that the correctness axis is, in this setting, substantially larger than

the cost axis. The gap between the best and worst policies compared here reaches 18 accuracy points and a six-fold difference in staleness, while the cost differences among the same policies are around 12 percent. Optimizing memory for size while treating currency as a secondary concern is optimizing the smaller of the two quantities.

Efficiency techniques need correctness audits, not only cost audits. Novelty gating reduces write-phase API cost by a factor of roughly 3.4 in its originating work [Wang et al., 2026]. This thesis shows that the same technique, applied to a workload in which facts get corrected, produces permanent, unrecoverable errors. This pattern, in which an efficiency technique carries an unbounded correctness liability that is invisible on any benchmark lacking the specific annotation needed to detect it, is likely to recur as agent memory systems continue to be optimized for cost. The general lesson is that any efficiency evaluation of a memory technique should include a workload specifically constructed to violate the optimization's underlying assumption, in the way this thesis's benchmarks were selected to include a genuine rate of factual correction.

Right-to-be-forgotten and auditability requirements point in the same direction as this thesis's central finding. A bitemporal store records, for every fact, when it was learned, when it stopped being true, and when it was ultimately removed. That is precisely the structure that regulatory deletion and data-provenance requirements need in order to be satisfied and audited, and it is the same structure that this thesis shows makes forgetting free of accuracy cost. Systems that treat memory as an undifferentiated vector store, without this structure, are likely to need to retrofit it for compliance reasons even if they never adopt it for the accuracy reasons demonstrated here.

Rigor at low cost is achievable. The framework built for this thesis is dependency-free, its entire pipeline runs offline and deterministically through a mock provider, its complete test suite finishes in a fraction of a second, and every call made to a real model provider is content-addressed and cached to disk. This is a demonstration that rigorous agent-memory research does not require a large compute budget to conduct, which has a direct bearing on who is able to participate in this line of research.

6.3 What DRIFT Means Going Forward

6.3.1 As an Artifact

DRIFT is not intended to compete with production memory libraries such as Mem0 or Zep as a memory store in its own right. It is better understood as a harness for choosing a forgetting policy before committing to one, together with a reference implementation of whichever policy that measurement process selects. The argument for why this choice needs to be measured rather than assumed is contained directly in the results: policy P3 was statistically tied with the eventual best policy on an early, uncontrolled campaign, and is the worst-performing policy of the seven on both fact-based benchmarks once the comparison is controlled, because the two situations differ in whether the underlying facts get corrected over time. A recommendation that happens to be correct for one workload can be actively harmful for another, and the only way to know which situation a given deployment is in is to measure it.

The intended usage pattern is to export an agent's real interaction history into DRIFT's schema, annotate the subset of questions whose answers have changed over the course of that history, run several candidate policies against the exported data, and deploy whichever policy the measurement selects. Adding a new policy to the framework requires only a subclass and a single registry entry, described in Section 3.2;

adding a new benchmark requires only a converter script and a configuration file.

6.3.2 Immediate Roadmap

Table 6.1 lists the highest-value remaining work, ordered by estimated value per unit of effort.

Table 6.1: Immediate roadmap, ordered by priority.

Priority	Work	Why	Cost
1	Complete the multi-agent campaign on MQuAKE-T	A 5-cell real-data pilot (Section 5.10.1) is complete and already surfaced a real finding: heterogeneous extraction fragments predicate strings and defeats P2's exact-match invalidation. Completing the remaining cells extends this from one topology to three and adds P4 and P5	16 cells
2	Judge pinning and per-record judge identity	Removes the confound identified in Section 5.9 from every future reported number	free
3	A conversational benchmark with recoverable supersession	Closes the extraction-noise gap, the largest remaining threat to validity; requires deterministic annotation recovery rather than an LLM annotator	new converter, 14 cells

continued on next page

Table 6.1 continued from previous page

Priority	Work	Why	Cost
4	Multi-hop MQuAKE-T	The existing converter already supports this via one flag; tests whether the ranking holds when the answer changes as an entailed consequence rather than by direct lookup	14 cells
5	Haystack-size ablation	Locating the crossover between retrieval-bound and policy-bound regimes would characterize when policy choice stops being the dominant factor	14 cells per size
6	`show_supersession` ablation	Answers whether the measured benefit lives in the policy itself or in explicitly telling the model about supersession	14 cells

6.3.3 Longer-Term Research Directions the Framework Enables

Learned forgetting policies. The three-hook policy interface described in Section 3.2 is, in effect, a policy API. A policy whose retrieval hook is a learned ranker, or whose maintenance hook is a learned pruner, drops into the same harness used to evaluate the seven hand-designed policies in this thesis and is compared under identical conditions. The seven policies evaluated here constitute a benchmark against which such learned policies could be measured directly.

Policy composition. Nothing in the design prevents combining hooks drawn from different policies, for example P2's retrieval-time filter together with P4's periodic language-model consolidation. The results reported in Chapter 5 suggest this combination is worth testing directly: P2 and P4 are statistically indistinguishable on MQuAKE-T accuracy while relying on entirely disjoint mechanisms, which raises the possibility that combining them captures benefits neither achieves alone.

Memory governance in multi-principal settings. GateMem frames shared-memory agent systems explicitly as a governance problem [Ren et al., 2026]. The `provenance` field carried by every fact in DRIFT's substrate, together with the cross-writer invalidation counter reported in Section 5.10, already provide the instrumentation needed to study whether forgetting policies need to be writer-aware in settings where multiple agents share one memory.

Adversarial memory. A superseded fact that is nonetheless retrieved is, in miniature, a small case of memory poisoning. The same substrate and the same staleness metric used throughout this thesis extend naturally to the study of injected false memories, where the relevant question becomes whether a forgetting policy can be made to remove adversarial writes without also removing legitimate corrections. The asymmetry between P2 and P3 documented in Section 6.1.2 suggests these two objectives may be in tension with one another, which is itself worth studying directly.

Deletion and compliance. The `t_pruned` and `provenance` fields together provide the audit trail that right-to-be-forgotten style requirements demand. Whether policy-driven forgetting of the kind studied in this thesis can be made to satisfy a formal deletion guarantee, in the sense used in the data-privacy literature, is an open question the substrate is well positioned to support studying.

6.4 Threats to Validity

The threats to the validity of the conclusions reported in this thesis are stated here in full, in order of severity, rather than selectively.

1. **Both real-data results come from fact corpora, not from natural conversation.** In both benchmarks, facts arrive as clean, atomic statements, so the extraction step never becomes a bottleneck for contradiction detection. In natural conversation it plausibly would. Policy P2 requires the extraction step to produce the identical subject and predicate for a fact and for the later fact that corrects it, and nothing in this thesis directly tests that requirement under the noisier extraction conditions natural conversation would introduce. This is, of the threats listed here, the single largest limit on how broadly the results generalize.

2. **MQuAKE-T's relations are narrow.** Of 1,868 candidate instances in the source dataset, 1,157 concern an edit to a head of government and 651 concern an edit to a head of state, yielding 96 distinct supersessions in the converted benchmark. MQuAKE-T is, in effect, a head-of-office corpus. It is well suited to testing real-versus-injected drift, which is the purpose it was selected for, but it does not test breadth across relation types.

3. **FactConsolidation's gold answers are deliberately counterfactual**, which makes its no-memory floor of 0.000 maximal by construction and makes the memory-versus-no-memory gap reported in Section 5.3 the largest gap the benchmark's design is capable of producing.

4. **Sample size is approximately 100 questions per benchmark.** The paired-difference intervals reported in Section 5.2 bound the P2-versus-P0 ac-

curacy gap to within roughly ±2.5 points, which is a meaningful equivalence claim at this sample size, but a true difference smaller than this bound would not be reliably detectable with the data collected here.

5. **Judge noise is 1 to 2 percent, with one 15 percent outlier cell**, as reported in Section 5.9. Differences smaller than roughly three points are not treated as interpretable in this thesis. This specifically affects the P2-versus-P4 comparison on MQuAKE-T.

6. **The NOMEM baseline of 0.000 is prompt-gated**, not direct evidence of the underlying model's parametric ignorance, as discussed in Section 3.3. Its staleness of 0.000 is achieved trivially, by refusing to answer at all.

7. **Policy P1's time-to-live parameter was set relative to synthetic session spacing**, specifically 20 days over 38 daily sessions. Its rank in the overall comparison is therefore partly a statement about that particular choice of parameter, and Section 5.7 shows in any case that its rank is workload-dependent regardless of the specific parameter chosen.

8. **RQ4 is only partially answered on real data.** A 5-cell pilot on one of three topologies (Section 5.10.1) found that heterogeneous extraction breaks P2's exact-match invalidation; the remaining sixteen cells, covering the other two topologies and policies P4 and P5, were not run, as discussed in Section 5.10.

9. **The `functional_predicates` setting was chosen per dataset** by direct inspection of each benchmark. Section 5.8 quantifies the consequence of this choice, namely that the setting controls all of P2's measured efficiency and none of its measured accuracy, but it remains a per-domain configuration burden that a deployment would need to repeat.

10. **Two answer models were used, both accessed through a single provider.** Policy rankings agree closely between the two models, as reported in Section 5.12, but both are recent, mid-size, instruction-tuned models, and neither is a frontier-scale model.

6.5 Limitations and Future Work

The threats listed in Section 6.4 translate directly into a work program. Three items would, if completed, most change this thesis's standing.

Close RQ4 on real data. One of the four research questions posed in Chapter 1 is validated in full only on 14 synthetic questions using a deterministic mock provider rather than a real language model. A 5-cell real-data pilot on the shared topology (Section 5.10.1) is complete and already changed the picture: it found that heterogeneous, multi-model extraction fragments predicate strings and defeats P2's exact-match contradiction detection, a boundary condition the single-agent experiments could not have surfaced. The harness needed to close the remaining gap is already built and already tested; the estimated cost is sixteen additional experimental cells, covering the independent and orchestrator topologies plus policies P4 and P5 on the shared topology.

Add a conversational regime. The extraction-noise threat listed first in Section 6.4 cannot be addressed by any amount of additional work on fact-corpus benchmarks, because the threat is specifically about what happens when facts must be extracted from natural conversation rather than presented as clean triples. Addressing it requires a conversational benchmark with recoverable supersession annotation, which does not currently exist in a usable form, and which an attempt described in

Section 4.1 to construct via an automated annotator on LongMemEval's knowledge-update subset failed to produce reliably. Building such a benchmark deterministically, rather than via an automated annotator, would be a contribution in its own right, independent of any subsequent experiment run on it.

Broaden the relation types tested. Both benchmarks used in this thesis are derived from Wikidata, and MQuAKE-T specifically is narrow in the sense described in Section 6.4. A workload containing genuinely multi-valued predicates, in contrast to the largely single-valued predicates both current benchmarks emphasize, would test whether the functional-predicate declaration this thesis relies on scales beyond the hand-curated lists used here.

Beyond these three priorities, the policy-composition direction, the learned-policy direction, and the adversarial-memory extension described in Section 6.3.3 are the directions the framework most directly supports as future work building on this thesis.

6.6 Conclusion

Forgetting is not a secondary concern in the design of language model agent memory. Across two benchmarks of different provenance, the choice of forgetting policy moves accuracy by up to 18 points and staleness by up to a six-fold factor, over a substrate, retrieval function, answer model, and judge held constant across every comparison. The cost differences among the policies compared in this thesis are around 12 percent. The correctness differences are an order of magnitude larger.

Four claims are supported by replicated, statistically tested evidence across both benchmarks used in this thesis.

1. Append-only memory measurably surfaces superseded facts, at a rate of 4.2 to

4.8 stale facts per retrieval, with staleness confidence intervals that exclude zero on both benchmarks.

2. Bitemporal contradiction invalidation is free of accuracy cost relative to append-only retention. Its accuracy difference from the append-only ceiling is bounded within roughly ± 2.5 points on one benchmark and ± 2.1 points on the other, while it spends 12 percent fewer tokens per question and stores 6.5 to 14.7 percent fewer facts.

3. The advantage measured for bitemporal invalidation is attributable to bitemporal semantics specifically, not to graph-aware structure in general. A structure-aware but deliberately time-blind ablation performs significantly worse, by 0.167 accuracy and 0.177 staleness on real-world knowledge updates, both at $p < 0.001$, with a legible underlying mechanism: structural corroboration count measures how established a fact is, not whether that fact is currently true.

4. Novelty-gated writing causes unrecoverable staleness, a result replicated across both an injected-supersession benchmark and a real-supersession benchmark, with the underlying mechanism demonstrated directly from saved knowledge-graph snapshots. The failure follows from the definition of a novelty gate itself, rather than from any particular implementation of one, which is what makes the claim transferable beyond this thesis's specific system.

Two methodological findings accompany these four claims. The configuration ablation decomposes P2's measured benefit into an accuracy component that holds unconditionally and an efficiency component that is contingent on a domain-specific declaration having been made correctly. The judge-reliability audit establishes a numerical noise floor for the evaluation methodology used throughout this thesis and

identifies a specific 12.5-point artifact on byte-identical answers that would, without the audit, very plausibly have been reported as a genuine policy effect.

The framework's own contribution, beyond any single reported number, is that the recommendations above are measurable rather than assumed. A policy that was statistically indistinguishable from the best policy on an early, uncontrolled campaign was, once the comparison was properly controlled, found to be the worst-performing policy on both fact-based benchmarks tested, because the two situations differed specifically in whether the underlying facts were subject to correction over time. Choosing a forgetting policy for a deployed agent without measuring that policy against the target workload is not a safe default, and this thesis's central practical contribution is that the cost of measuring it, rather than assuming it, is now approximately an afternoon of work.

Appendix A

Complete Per-Cell Results and Significance Matrix

This appendix contains the complete per-cell results underlying every summary table in Chapter 5, together with the full pairwise significance matrix across all twenty-one policy pairs, both metrics, and both benchmarks. Every value in this appendix is drawn directly from the result files produced by an experimental run and can be regenerated exactly using the commands given in Appendix B.

A.1 FactConsolidation, 14 Cells

Table A.1 reports every cell of the FactConsolidation matrix: seven policies, each evaluated with two answer models. Accuracy confidence intervals are percentile bootstrap at the 95 percent level, 5,000 resamples, computed per cell with $n = 100$.

Table A.1: FactConsolidation, all 14 cells.

Policy	Model	Acc.	95% CI	F1	Stale.	Tok/q	Active	Inval	Pruned	Rej.	R-inv
NOMEM	MiniMax-M3	0.000	[.000,.000]	0.000	0.000	316	380	92	0	0	0.00
NOMEM	GLM-5.2	0.000	[.000,.000]	0.000	0.000	184	380	92	0	0	0.00
P0	MiniMax-M3	0.820	[.740,.890]	0.555	0.060	871	380	92	0	0	4.20
P0	GLM-5.2	0.860	[.790,.930]	0.644	0.000	754	380	92	0	0	4.20
P1	MiniMax-M3	0.610	[.510,.710]	0.382	0.030	842	210	27	170	0	2.08
P1	GLM-5.2	0.650	[.550,.740]	0.461	0.010	756	210	27	170	0	2.08
P2	MiniMax-M3	0.830	[.750,.900]	0.490	0.050	772	324	36	56	0	0.00
P2	GLM-5.2	0.850	[.770,.920]	0.608	0.000	657	324	36	56	0	0.00
P3	MiniMax-M3	0.650	[.560,.740]	0.413	0.160	865	334	67	0	46	3.91
P3	GLM-5.2	0.680	[.580,.770]	0.528	0.120	748	334	67	0	46	3.91
P4	MiniMax-M3	0.780	[.700,.860]	0.530	0.100	874	374	92	6	0	4.24
P4	GLM-5.2	0.820	[.740,.890]	0.611	0.020	753	374	92	6	0	4.24
P5	MiniMax-M3	0.770	[.680,.850]	0.515	0.070	870	365	87	15	0	4.16
P5	GLM-5.2	0.820	[.740,.890]	0.643	0.000	754	365	86	15	0	4.12

A.2 MQuAKE-T, 14 Cells

Table A.2 reports every cell of the MQuAKE-T matrix, sized identically to Table A.1, with $n = 96$ per cell.

Table A.2: MQuAKE-T, all 14 cells.

Policy	Model	Acc.	95% CI	F1	Stale.	Tok/q	Active	Inval	Pruned	Rej.	R-inv
NOMEM	MiniMax-M3	0.000	[.000,.000]	0.003	0.000	311	382	59	0	0	0.00
NOMEM	GLM-5.2	0.000	[.000,.000]	0.003	0.000	156	382	59	0	0	0.00
P0	MiniMax-M3	0.969	[.927,1.000]	0.622	0.094	897	382	59	0	0	4.80
P0	GLM-5.2	0.969	[.927,1.000]	0.973	0.010	734	382	59	0	0	4.80
P1	MiniMax-M3	0.875	[.802,.938]	0.569	0.031	885	251	39	131	0	3.28
P1	GLM-5.2	0.865	[.792,.927]	0.870	0.021	729	251	39	131	0	3.28
P2	MiniMax-M3	0.969	[.927,1.000]	0.623	0.062	791	357	34	25	0	0.00
P2	GLM-5.2	0.969	[.927,1.000]	0.990	0.010	641	357	34	25	0	0.00
P3	MiniMax-M3	0.781	[.698,.865]	0.442	0.177	883	351	44	0	31	3.94
P3	GLM-5.2	0.802	[.719,.875]	0.832	0.146	727	351	44	0	31	3.94
P4	MiniMax-M3	0.948	[.896,.990]	0.628	0.094	883	374	56	8	0	4.79
P4	GLM-5.2	0.958	[.917,.990]	0.960	0.021	725	369	53	13	0	4.64
P5	MiniMax-M3	0.781	[.698,.865]	0.537	0.260	907	365	57	17	0	5.43
P5	GLM-5.2	0.823	[.750,.896]	0.819	0.167	725	365	56	17	0	5.31

A.3 MQuAKE-T Ablation, 3 Cells

Table A.3 reports the three cells of the configuration ablation described in Section 5.8, run with `functional_predicates` omitted so that the substrate default applies and covers none of MQuAKE-T's relations. The P0 accuracy value marked as an artifact is discussed in full in Section 5.9.

Table A.3: MQuAKE-T ablation, 3 cells, GLM-5.2 only.

Policy	Accuracy	F1	Staleness	Tok/q	Active	Inval	Rejected
P0	0.844 (artifact, true 0.969)	0.973	0.010	734	382	1	0
P2	0.969	0.980	0.010	733	382	1	0
P3	0.802	0.832	0.146	727	351	0	31

A.4 LoCoMo, Partial

The LoCoMo matrix was two of thirty planned cells complete at the time of writing and is reported here for the contrast between regimes only, and explicitly not as a result to be interpreted alongside Sections A.1 and A.2.

Table A.4: LoCoMo, partial, 2 of 30 cells.

Policy	Model	Accuracy	Staleness	Tok/q	Note
NOMEM	MiniMax-M3	0.037	0.000	311	staleness cannot fire; 0/300 annotated
NOMEM	GLM-5.2	0.037	0.000	183	as above

Per-category NOMEM accuracy on LoCoMo: temporal 0.000, single-hop 0.031, multi-hop 0.047, adversarial 0.060, open-domain 0.067. Unlike the two fact-corpus benchmarks used elsewhere in this thesis, NOMEM accuracy is non-zero here, because LoCoMo's questions concern a fictional conversation the underlying model can occasionally answer correctly by guessing from conversational context alone, without needing memory at all.

A.5 Complete Pairwise Significance Matrix

Tables A.5 and A.6 report all 21 policy pairs, both metrics, on both benchmarks. Counts are discordant pairs from the paired McNemar test; p is the exact two-sided binomial probability. Significance is marked as $*$ for $p < 0.05$, $**$ for $p < 0.01$, and $***$ for $p < 0.001$.

Table A.5: Complete pairwise significance matrix, FactConsolidation.

Comparison	Accuracy	p	Staleness	p
NOMEM vs P0	0-168	< 0.001 ***	0-6	0.031 *
NOMEM vs P1	0-126	< 0.001 ***	0-4	0.125
NOMEM vs P2	0-168	< 0.001 ***	0-5	0.062
NOMEM vs P3	0-133	< 0.001 ***	0-28	< 0.001 ***
NOMEM vs P4	0-160	< 0.001 ***	0-12	< 0.001 ***
NOMEM vs P5	0-159	< 0.001 ***	0-7	0.016 *
P0 vs P1	44-2	< 0.001 ***	4-2	0.688
P0 vs P2	**3-3**	**1.000**	**5-4**	**1.000**
P0 vs P3	36-1	< 0.001 ***	1-23	< 0.001 ***
P0 vs P4	8-0	0.008 **	1-7	0.070
P0 vs P5	9-0	0.004 **	0-1	1.000
P1 vs P2	1-43	< 0.001 ***	2-3	1.000
P1 vs P3	34-41	0.489	2-26	< 0.001 ***
P1 vs P4	5-39	< 0.001 ***	2-10	0.039 *
P1 vs P5	9-42	< 0.001 ***	1-4	0.375
P2 vs P3	36-1	< 0.001 ***	1-24	< 0.001 ***
P2 vs P4	11-3	0.057	4-11	0.118
P2 vs P5	11-2	0.022 *	3-5	0.727
P3 vs P4	9-36	< 0.001 ***	24-8	0.007 **
P3 vs P5	8-34	< 0.001 ***	23-2	< 0.001 ***
P4 vs P5	9-8	1.000	7-2	0.180

Table A.6: Complete pairwise significance matrix, MQuAKE-T.

Comparison	Accuracy	p	Staleness	p
NOMEM vs P0	0-186	< 0.001 ***	0-10	0.002 **
NOMEM vs P1	0-167	< 0.001 ***	0-5	0.062
NOMEM vs P2	0-186	< 0.001 ***	0-7	0.016 *
NOMEM vs P3	0-152	< 0.001 ***	0-31	< 0.001 ***
NOMEM vs P4	0-183	< 0.001 ***	0-11	< 0.001 ***
NOMEM vs P5	0-154	< 0.001 ***	0-41	< 0.001 ***
P0 vs P1	20-1	< 0.001 ***	6-1	0.125
P0 vs P2	**2-2**	**1.000**	**6-3**	**0.508**
P0 vs P3	38-4	< 0.001 ***	4-25	< 0.001 ***
P0 vs P4	5-2	0.453	3-4	1.000
P0 vs P5	34-2	< 0.001 ***	2-33	< 0.001 ***
P1 vs P2	1-20	< 0.001 ***	2-4	0.688
P1 vs P3	35-20	0.058	2-28	< 0.001 ***
P1 vs P4	6-22	0.004 **	2-8	0.109
P1 vs P5	32-19	0.092	0-36	< 0.001 ***
P2 vs P3	38-4	< 0.001 ***	4-28	< 0.001 ***
P2 vs P4	7-4	0.549	4-8	0.388
P2 vs P5	**34-2**	< 0.001 *******	**3-37**	< 0.001 *******
P3 vs P4	6-37	< 0.001 ***	24-4	< 0.001 ***
P3 vs P5	23-25	0.885	19-29	0.193
P4 vs P5	36-7	< 0.001 ***	5-35	< 0.001 ***

NOMEM appearing to outperform every other policy on staleness in both tables is an artifact of refusing to answer rather than evidence of superior memory

hygiene: a policy that never answers cannot repeat a stale value, and staleness is only interpretable as a measure of policy quality among the policies that actually attempt an answer.

Appendix B

Figures, Dataset Schema, Reproduction, and Integration

B.1 Figure Index

Every figure in this thesis is a 300 dpi PNG regenerated automatically by the analysis code from the underlying result files. Table B.1 lists every figure used, its role, and the chapter or section it supports.

Table B.1: Figure index.

Figure	Shows	Supports
Architecture	Pipeline diagram from session text to judged answer	Section 3.2
Accuracy and staleness	Grouped bars per policy	Section 5.1
Trade-off scatter	Accuracy plotted against staleness, one point per policy	Section 5.1
Memory growth	Active fact count over sessions	Section 5.11
Token cost	Mean tokens per question	Section 5.4
Model comparison	Per-model policy ranking	Section 5.12
Policy radar	All seven policies across every metric simultaneously	Section 5.1
Baseline versus DRIFT	NOMEM and P0 plotted against the five forgetting policies	Section 5.3
Swarm topology	Per-topology comparison across policies	Section 5.10
Swarm variance	Mean and standard deviation across independent agents	Section 5.10

Beyond the static figures included in this thesis, the codebase also produces several interactive, browser-based artifacts that are not reproducible on paper but are referenced here for completeness: a self-contained dashboard covering every benchmark, a scrubbable bitemporal memory replay that shows the novelty-gating failure mode from Section 5.6 as a timeline of events rather than as a static table, and per-cell

knowledge-graph snapshots and replays for every experimental cell. All are produced by the `make figures record dashboard replay` command sequence given in Section B.3.

B.2 The Dataset Schema

Any workload can be evaluated using DRIFT by emitting it in the following structure. This is the schema both benchmarks used in this thesis were converted into.

```
{"conversations": [{
  "id": "user-42",
  "sessions": [{"time": 1704067200.0,
                "lines": ["user: I moved to Seattle.",
                          "assistant: Noted."]}],
  "questions": [{"question": "Where does the user live?",
                 "answer": "Seattle",
                 "category": "knowledge_update",
                 "stale_answer": "Boston"}]
}]}
```

The `stale_answer` field is what the staleness metric defined in Section 3.4 depends on. Accuracy can be computed without it, but staleness cannot fire in its absence, which is exactly the structural gap identified for LoCoMo and LongMemEval in Section 4.1. When adapting a new workload to this schema, the recommendation stated in that section applies directly: recover this annotation deterministically from the workload's own structure wherever possible, rather than relying on an automated annotator.

B.3 Reproducing Every Number in This Thesis

The following command sequence, run from a clean checkout of the codebase, reproduces every number reported in Chapter 5 and every figure listed in Table B.1.

```
make install       # editable install, dev + analysis extras
make test          # 99 offline tests, no API keys, no network

make fc-data && make fc     # FactConsolidation, 14 cells
make mq-data && make mq     # MQuAKE-T, 14 cells
make mq-ablation             # configuration probe, 3 cells

make figures record dashboard
make compare        # cross-dataset comparison table
make replay          # novelty gating as a scrubbable timeline
make swarm           # multi-agent topologies, offline
```

Real-API runs require provider keys placed in a local `.env` file, following the template in `.env.example`. Every call made to a model provider during any of these runs is disk-cached by the SHA-256 hash of its inputs, as described in Section 4.4, so a rerun of an already-completed campaign costs nothing and produces byte-for-byte identical output to the original run.

B.4 Using DRIFT in Another Agent

The framework built for this thesis is released as an installable Python package with a short integration surface, so that the practical recommendation given in Section 6.2.2

can be tested directly on a reader's own workload rather than taken on the authority of this thesis alone. The following listing shows a minimal integration.

```
import time
from drift import Memory

mem = Memory(
    policy="P2",
    provider="openrouter:z-ai/glm-5.2",
    embedder="ollama:nomic-embed-text",
    functional_predicates=["lives_in", "works_at",
                           "assigned_to", "status"],
)

mem.observe(["user: I moved to Seattle."], when=time.time())
memory_block = mem.context("Where does the user live?", k=8)
```

A dedicated test in the codebase pins that this integration facade produces fact-for-fact identical decisions to the full experimental harness when run under the same policy, which is precisely what licenses reading the measured results reported in Chapter 5 as practical advice about the library itself, rather than as results that apply only to the specific harness used to produce them.

Adding a new policy to the framework, for a reader who wishes to test a policy not among the seven compared in this thesis, requires only a subclass and a single line registering it:

```
class P6MyPolicy(Policy):
```

```
    name = "P6_my_policy"
    def on_write(self, fact, graph) -> bool:
        return True
    def after_session(self, graph, provider, now):
        ...
    def on_retrieve(self, facts, now):
        return facts

POLICY_REGISTRY["P6"] = P6MyPolicy
```

Bibliography

Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. STALE: Can LLM agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.

Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. In *The Fourteenth International Conference on Learning Representations*, 2026. arXiv:2507.05257; source of the FactConsolidation split.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Vedant Patel. Supersede: Diagnosing and training the memory-update gap in LLM agents. *arXiv preprint arXiv:2606.27472*, 2026.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

Zhe Ren, Yibo Yang, Yimeng Chen, Zijun Zhao, Benshuo Fu, Zhihao Shu, Bingjie Zhang, Yangyang Xu, Dandan Guo, and Shuicheng Yan. GateMem: Benchmarking memory governance in multi-principal shared-memory agents. *arXiv preprint arXiv:2606.18829*, 2026.

Sijia Wang, Dhanajit Brahma, and Ricardo Henao. SAGE: A novelty gate for efficient memory evolution in agentic LLMs. *arXiv preprint arXiv:2605.30711*, 2026.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

Dylan Zhang, Yanshan Lin, Zhengkun Wu, Yihang Sun, Bingxuan Li, Dianqi Li, and Hao Peng. Useful memories become faulty when continuously updated by LLMs. *arXiv preprint arXiv:2605.12978*, 2026.

Zexuan Zhong, Zhengxuan Wu, Christopher D. Manning, Christopher Potts, and

Danqi Chen. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.14795; source of the MQuAKE-T split.